



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN III

FRONT END

Unidad Temática N° 1:
HTML, CSS y JavaScript

Teórico
2° Año – 3° Cuatrimestre



Índice

1. Introducción y objetivos de aprendizaje	5
2. Herramientas del Desarrollador: Node.js y NPM	5
2.1. El Modelo Mental: Node.js vs El Navegador	5
2.2. Instalación Profesional y Gestión de Versiones	6
2.3. Ecosistema NPM en el Día a Día	7
2.4. Anatomía del package.json y el Archivo Lock	7
SemVer (Versionado Semántico):	8
2.5. Seguridad Básica en 2026	9
2.6. Limpieza Segura: La Estrategia de "Reset"	9
2.7. El Puente Directo a Angular	10
2.8. Errores Comunes (Los "Mata-Proyectos")	10
3. HTML Esencial: Semántica, Accesibilidad (A11y) y Renderizado	11
3.1. ¿Cómo se renderiza HTML en el navegador?	11
3.2. Relación de Archivos y Rutas (URLs)	12
3.3. Semántica HTML5 y SEO	12
3.4. Accesibilidad (A11y) y Estándar WCAG 2.2	13
1. Estructura de Encabezados (Headings):	13
2. Navegación por Teclado y Foco:	14
3. Formularios e Imágenes:	14
4. Roles y ARIA (Lo esencial):	14
3.5. Plantilla Base (Boilerplate) Profesional	15
Errores Comunes	16
4. CSS Esencial: Layouts Modernos y Tailwind	16
4.1 ¿Qué es CSS y cómo funciona en la Web?	16
4.2 El Modelo de Caja (Box Model)	17
4.2 Layouts Modernos: Flexbox	18
4.4 Layouts Modernos: CSS Grid	19
4.5 Subgrid: La Revolución	19
4.6 Tailwind CSS: El Framework Definitivo	20
4.7 Instalación y Configuración Profesional con Vite	21

4.8 Documentación de Referencia (Tailwind Docs):	22
5. JavaScript: Fundamentos, Arquitectura y Calidad de Código	23
5.1 Fundamentos del Lenguaje (Variables, Tipos y Operadores)	23
5.2 Funciones, Contexto y Closures	24
Closures (Clausuras): Memoria Fotográfica	25
5.3 El Modelo de Ejecución y el Event Loop	25
5.4 Memoria, Inmutabilidad y Copias	26
5.5 Objetos, Prototipos y Clases (OOP en JS)	27
5.6 Módulos ES (Import / Export)	28
5.7 Errores y Manejo de Excepciones	28
5.8 Buenas Prácticas de Calidad de Código (Clean Code)	29
5.9 Seguridad Básica: Prevención de XSS	29
5.10 Errores Típicos	29
6. DOM (Document Object Model): Estructura, Render y Rendimiento	30
6.1. El Ciclo de Renderizado del Navegador (Critical Rendering Path).....	30
6.2. Selección y Navegación Moderna (Traversing)	31
6.3. Modificación, Atributos y APIs 2026.....	32
6.4. Rendimiento Extremo: El Terror del "Layout Thrashing"	32
6.5. Creación Eficiente y Renderizado de Listas	33
6.6. Seguridad: textContent vs innerHTML (Riesgo XSS)	34
6.7. El DOM y la Accesibilidad (A11y)	34
6.8. El Ciclo de Vida: DOMContentLoaded vs load	34
6.9. Errores Comunes (Los "Mata-Proyectos")	35
7. Manejo de Eventos Avanzado y Rendimiento	36
7.1. El Objeto event y Control de Propagación	36
7.2. El Flujo del Evento (Event Flow)	36
7.3. Clasificación de Eventos 2026 (Tabla de Referencia)	37
7.4. Pointer Events y Accesibilidad por Teclado	38
7.5. Delegación de Eventos (El Patrón Rey).....	38
7.6. Rendimiento: Throttle, Debounce y Opciones Pasivas.....	39
7.7. Fugas de Memoria (Memory Leaks) y AbortController	39
7.8. Custom Events (Desacoplar Lógica).....	40
7.9. Errores Comunes en Eventos	40

8. Asincronía y Fetch API	41
8.1. El Modelo Asíncrono y el Event Loop (Repaso aplicado)	41
8.2. Promesas en Profundidad	41
8.3. Fetch API	42
8.4. AbortController, Timeouts y Cancelaciones	44
8.5. CORS Explicado para Frontend	45
8.6. Buenas Prácticas de UX (Experiencia de Usuario)	46
8.7. Seguridad Mínima en Fetch	46
9. Web Storage API y Privacidad (Cookies)	47
9.1. El Modelo Mental y Reglas de Oro	47
9.2. LocalStorage vs SessionStorage	47
9.3. Robustez y Errores Típicos	48
9.4. Sincronización entre Pestañas (Evento 'storage')	48
9.5. Alternativas de Almacenamiento Modernas	49
9.6. Cookies a Fondo	49
9.7. Privacidad Moderna: CHIPS y Cookies Particionadas	50
9.8. ¿Dónde guardo este dato?	51
10. Práctica Integradora: "TaskFlow Profesional"	51
Paso 0: Setup Profesional del Proyecto	52
Paso 1: HTML Semántico y Accesible (index.html)	52
Paso 2: Persistencia y "Fake API" (src/api.js y src/storage.js)	54
Paso 3: Renderizado Eficiente del DOM (src/dom.js)	55
Paso 4: El Orquestador (src/app.js)	56
Paso 5: Debugging (DevTools)	57
11. Errores Comunes de Desarrollo	57

1. Introducción y objetivos de aprendizaje

El desarrollo frontend en 2026 se define por la **orquestración de capacidades nativas**. Hemos superado la era de **dependencia absoluta** de **librerías externas** para tareas básicas; hoy, el **navegador** es un **entorno de ejecución maduro** que, junto con el ecosistema de **Node.js**, permite construir aplicaciones altamente **eficientes, accesibles y modulares**.

Esta unidad sienta las **bases técnicas** para el resto de la tecnicatura. Comenzaremos con la **estructura** y el **estilo esencial**, exploraremos el **entorno de herramientas profesional (Node/ NPM)** y profundizaremos en el corazón de la **interactividad**: la **manipulación del DOM** y la **gestión de eventos**.

Objetivos de aprendizaje:

- Estructurar documentos mediante **HTML semántico** y criterios de **accesibilidad WCAG 2.2**.
- Implementar **layouts modernos** con **CSS Grid, Subgrid y Flexbox**.
- Gestionar **dependencias y scripts de automatización** mediante **Node.js** y **NPM**.
- Dominar la **manipulación profunda del DOM** y el **flujo avanzado de Eventos (Delegación)**.
- Implementar **comunicación asíncrona robusta** con **Fetch API** y **AbortController**.
- Gestionar la **persistencia de datos** en el cliente y entender el nuevo marco de **privacidad de cookies**.

2. Herramientas del Desarrollador: Node.js y NPM

Aunque el código **frontend** se ejecutará finalmente en el **navegador web** del usuario, el flujo de trabajo profesional, la preparación del código y el **ecosistema de herramientas** ocurren casi en su totalidad dentro de **Node.js**. Entender **Node** y su **gestor de paquetes (NPM)** es el puente indispensable hacia **frameworks estructurados** como **Angular**.

2.1. El Modelo Mental: Node.js vs El Navegador

Para un **desarrollador Frontend**, es **vital** entender dónde se ejecuta el **código**. Tanto **Node.js** como **Chrome** usan el mismo **motor interno (V8 de Google)** para ejecutar **JavaScript**, pero sus **entornos** y **superpoderes (APIs disponibles)** son completamente **distintos**.

- El **Navegador (Entorno de UI)**: **JS** aquí tiene acceso al **DOM** (*document*), a la **ventana** (*window*), y a **APIs web** como *fetch* o *localStorage*. Su objetivo es renderizar **interfaces**. No puede tocar el **disco duro** del usuario por **seguridad (Sandbox)**.
- **Node.js (Entorno de Tooling/Servidor)**: **JS** aquí **NO** tiene **DOM** ni **window**. En su lugar, tiene acceso al **Sistema Operativo**: puede **leer/escribir archivos** nativamente (**módulo fs**), levantar **servidores de red** y ejecutar **scripts** en **consola**.

¿Para qué usamos **Node.js** en **Frontend** si no hacemos **Backend**?

1. Herramientas de Construcción (Build Tools): El navegador no entiende TypeScript, SASS, ni Tailwind nativamente. Node.js ejecuta programas (como Vite o Angular CLI) que traducen ("transpilan") y empaquetan tu código moderno a un HTML/CSS/JS estándar y minificado.
2. Servidores de Desarrollo Locales: Levanta servidores (ej. *localhost:4200*) con HMR (Hot Module Replacement), que actualizan tu pantalla en milisegundos cada vez que guardas un archivo, sin recargar la página completa.
3. Calidad de código: Ejecuta linters (analizadores estáticos) y frameworks de testing.

2.2. Instalación Profesional y Gestión de Versiones

En el ámbito **profesional** (y universitario), no basta con descargar **Node** dando clic en "Siguiente, siguiente, instalar". Los **frameworks robustos** (como **Angular**) requieren **versiones específicas de Node.js**.

- **LTS (Long Term Support)**: Siempre debes instalar versiones LTS (números pares, ej. v20, v22), que garantizan estabilidad y actualizaciones de seguridad por años.
- **Gestores de Versiones (nvm / fnm)**: Herramientas como Node Version Manager (nvm en Mac/Linux) o fnm te permiten tener instaladas múltiples versiones de Node en tu PC y cambiar entre ellas con un simple comando (ej. `nvm use 20`). Esto evita el colapso absoluto cuando tienes un proyecto viejo en Angular 14 y uno nuevo en Angular 18 que exigen distintas versiones del motor.

2.3. Ecosistema NPM en el Día a Día

NPM (Node Package Manager) es la base de datos pública de librerías más grande del mundo y tu herramienta de consola principal.

Comandos Clave de **NPM**:

- **npm init -y**: Inicializa un proyecto vacío creando rápidamente el archivo **package.json**.
- **npm install <paquete>**: Descarga una **librería** y la anota como **dependencia**.
- **npm install** (a secas): Lee el **package.json** y descarga absolutamente todo lo necesario (**vital al clonar un repositorio**).
- **npm ci** (**Clean Install**): Usado en **servidores de despliegue (CI/CD)**. Borra todo e instala las **versiones exactas** del **archivo lock** de manera **estricta y limpia**.
- **npx**: **Ejecución ágil** sin contaminación

En el pasado, herramientas como **Angular CLI** se instalaban **globalmente** en tu PC (**npm install -g**). Hoy, esto se evita debido al **version drift** (tu PC tiene una versión global desactualizada, mientras el proyecto pide otra).

En su lugar, usamos **npx <comando>** (ej. **npx tailwindcss init**). Esto descarga la herramienta **temporalmente**, la ejecuta y la elimina, garantizando que siempre uses la **versión correcta** sin ensuciar tu **sistema operativo**.

2.4. Anatomía del package.json y el Archivo Lock

El **package.json** es el "**DNI**" de tu **aplicación**. Define su **identidad**, cómo se **arranca** y qué **necesita** para **vivir**.

```
{
  "name": "mi-proyecto-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "eslint .",
    "test": "jest"
  },
  "dependencies": {
    "@angular/core": "^18.0.0",
    "chart.js": "^4.4.0"
  },
  "devDependencies": {
    "tailwindcss": "^3.4.1",
    "typescript": "~5.3.3"
  }
}
```

- **type: "module": (Estándar moderno)** Indica que Node tratará tus archivos usando la sintaxis actual de **import/export (ES Modules)** en lugar del viejo sistema **require()**.
 - **scripts: Atajos para comandos complejos.** Ejecutas **npm run dev** en tu consola en lugar de memorizar rutas larguísimas de ejecución.
 - **dependencies vs devDependencies:**
- Las **dependencias de producción** viajan al **cliente final**. Tu código depende de ellas para ejecutarse en el **navegador** (ej: Angular, Chart.js, librerías de UI).
 - Las **dependencias de desarrollo** mueren en tu **computadora**. Jamás llegan al **usuario final** (ej: Tailwind CSS para generar el diseño, **TypeScript**, herramientas de Testing).

SemVer (Versionado Semántico):

Los símbolos junto a las versiones (^, ~) siguen la norma SemVer (Mayor.Menor.Parche). El símbolo ^ (ej. ^18.0.0) permite actualizaciones automáticas de "parches" y versiones "menores" (sin romper compatibilidad), pero bloquea saltos a versiones Mayores (ej. no saltará a v19).

El intocable package-lock.json:

Si package.json dice "Instala la librería X v1.0 o superior", el package-lock.json anota el hash de seguridad y la versión exacta al milímetro que se instaló en tu PC hoy (ej. 1.0.4).

Regla estricta: Este archivo jamás se edita a mano, pero siempre se sube al repositorio Git. Esto asegura que si tu compañero clona el proyecto 6 meses después, instale exactamente los mismos bytes que tú, previniendo el infame problema de "en mi máquina sí funciona".

2.5. Seguridad Básica en 2026

Instalar dependencias implica meter código de extraños a tu proyecto. Los ataques de cadena de suministro (Supply Chain attacks) son reales.

- Criterio de Selección: No instales librerías aleatorias o abandonadas. Verifica su fecha de última actualización, issues abiertos y descargas semanales.
- npm audit: Comando nativo que escanea tu árbol de dependencias buscando vulnerabilidades publicadas en bases de datos de ciberseguridad y te sugiere cómo arreglarlas (npm audit fix).

2.6. Limpieza Segura: La Estrategia de "Reset"

El agujero negro llamado **node_modules** es la **carpeta** donde se descarga físicamente el **código** de las **librerías**. Pesa muchísimo y jamás se sube a **Git** (debe estar en el **.gitignore**).

A veces, la **caché de dependencias** se corrompe o hay **conflictos** al cambiar de rama en Git.

El proceso correcto de **Reset**: Si tu proyecto arroja **errores incomprensibles** de módulos faltantes, NO borres el **package.json**. La secuencia de rescate definitiva es:

1. **Elimina** la carpeta **node_modules/** completa.
2. **Elimina** el archivo **package-lock.json**.
3. Ejecuta **npm install** nuevamente. Esto forzará una **reconstrucción limpia** de todo tu **árbol de dependencias**.

2.7. El Puente Directo a Angular

Esta unidad es el **cimiento absoluto** para **Angular**, un **framework** que respira este **ecosistema**:

- **Angular CLI (ng)** es un **paquete** manejado y ejecutado vía **Node/NPM**.
- **Angular** está escrito en **TypeScript**, un **compilador** que corre sobre **Node**.
- Cuando ejecutas **ng serve** o **ng build**, estás disparando **arquitecturas de Node** bajo el capó que traducen y empaquetan tu **lógica** en **componentes eficientes**. Sin **Node.js**, **Angular** (y el **Frontend moderno** en general) no existe.

2.8. Errores Comunes (Los "Mata-Proyectos")

1. **npm o node no reconocidos en consola (Problema de PATH)**: Ocurre generalmente en **Windows** al instalar **Node** y no reiniciar la **consola**, o si el instalador no agregó **Node** al **PATH** del sistema. Solución: Reinstalar marcando explícitamente **"Add to PATH"** y **reiniciar** completamente el **editor** (ej. VS Code).
2. **Error de clonación masivo ("Faltan módulos")**: El alumno descarga un **proyecto** (como un clon base de Angular), intenta ejecutar **npm run dev** y la **consola** colapsa. Causa: Olvidó **"hidratar"** el proyecto. Recordar que los **repositorios** no incluyen **node_modules**. Solución: Siempre que **clones** un proyecto, tu primer comando debe ser **npm install**.
3. **Confusión de Entornos (Window no está definido)**: Intentar correr un archivo **script.js** (que contiene **document.querySelector**) en la **consola** usando el comando **node script.js**. Causa: **Node.js** no tiene **DOM** ni **interfaz gráfica**. Solución: El **JS** que interactúa con **HTML** debe ser **enlazado** en un archivo **.html** y abierto en un **navegador**.
4. **Conflicto Severo de Versiones de Node**: Un **proyecto Angular** crashea al inicio indicando **Unsupported Engine**. Causa: Tu máquina tiene **Node v16** y el proyecto exige **v20**. Solución: Usar un **gestor** como **fnm** o **nvm** para instalar la **versión correcta** y **alternar**.
5. **npm install en la carpeta equivocada**: Clonar un **proyecto** que está dentro de una **subcarpeta** (ej. **/unidad1/mi-app**) y ejecutar **npm**

`install` estando situado en la **raíz** `/unidad1`. Creará **dependencias "huérfanas"** en la **raíz**. Solución: Asegúrate de verificar tu **ruta actual** en la consola (`cd mi-app`) antes de **instalar**.

6. **Mezclar Gestores de Paquetes:** Usar `npm install` hoy y mañana `yarn add` o `pnpm install` en el mismo proyecto por seguir un tutorial ciegamente. Esto genera múltiples archivos **lock** (`yarn.lock` y `package-lock.json`) causando **colisiones destructivas** en la **resolución de paquetes**. Escoge uno y **respétalo**.
7. **Olvidar reiniciar el Servidor de Desarrollo:** Instalar un **nuevo paquete** (`npm install algo`) o modificar la **configuración estructural** (como `tailwind.config.js`), y frustrarse porque los cambios no se ven en pantalla. Solución: El **servidor de desarrollo** a veces no detecta **cambios estructurales**. Corta el proceso en **consola** (**Ctrl + C**) y vuelve a levantarlo (`npm run dev`).

3. HTML Esencial: Semántica, Accesibilidad (A11y) y Renderizado

HTML (HyperText Markup Language) no es un lenguaje de programación; es un lenguaje de marcado. Su única función es describir la estructura y el significado de la información.

3.1. ¿Cómo se renderiza HTML en el navegador?

Es **fundamental** entender que el **navegador** no pinta directamente tu archivo `.html` en la pantalla. Realiza un proceso complejo conocido como **Critical Rendering Path** (Ruta Crítica de Renderizado):

1. **Análisis (Parsing):** El navegador descarga el archivo de **texto** `.html` y lo lee línea por línea.
2. **Construcción del DOM (Document Object Model):** A medida que lee las **etiquetas** (`<body>`, `<h1>`, `<p>`), las convierte en "**nodos**" u "**objetos**" en la memoria de la computadora. A este **árbol genealógico virtual** se le llama **DOM**.
3. **Petición de Recursos:** Si el HTML encuentra etiquetas `<link rel="stylesheet">` o ``, el navegador pausa o paraleliza la descarga de esos **archivos externos**. Construye el **CSSOM (modelo de CSS)**.

4. **Render Tree:** El navegador **fusiona** el **DOM** y el **CSSOM**, descartando lo **invisible** (ej. elementos con `display: none`).
5. **Layout y Paint:** Finalmente, calcula la matemática de dónde va cada **caja** en la pantalla y "**pinta**" los **píxeles**.

Analogía (El Plano Arquitectónico): Imagina que **HTML** es el **plano estructural** de una casa (paredes, marcos de puertas). El **navegador** es el capataz que lee el plano y construye la **estructura física** (el **DOM**). **JS** sería la **instalación eléctrica** que le da vida (abre y cierra puertas) y **CSS** la **pintura y decoración**.

3.2. Relación de Archivos y Rutas (URLs)

El archivo **HTML** es el **director de orquesta** de un proyecto web. Nunca está solo; necesita **importar** otros **recursos** para crear una experiencia completa. Para esto, utiliza **Rutas (Paths)**.

- HTML llama a **CSS**: `<link rel="stylesheet" href="estilos/main.css">` (En el `<head>`).
- HTML llama a **JS**: `<script src="scripts/app.js" defer></script>` (Al final del `<body>` o en el `<head>` con **defer** para no **bloquear el renderizado**).
- HTML llama a **Imágenes**: ``.

Rutas Relativas vs Absolutas:

- **Absolutas:** Incluyen el **protocolo** y **dominio completo**. Van a buscar algo a **otro servidor**. (Ej. <https://mi-api.com/logo.jpg>).
- **Relativas:** Buscan archivos **dentro de la misma carpeta** del proyecto. (Ej. `./assets/logo.png` significa "en la misma carpeta donde estoy, entra a `assets`"). Comprender esto evita el clásico **error** de "**la imagen no carga**".

3.3. Semántica HTML5 y SEO

La **Semántica** implica usar la etiqueta correcta para el trabajo correcto. Un `<div>` no tiene significado (es solo un contenedor genérico), mientras que un `<button>` grita "**¡Soy interactivo!**".

¿Por qué importa la semántica?

1. **Mantenibilidad:** El código es más fácil de leer para otros **desarrolladores**.

2. **SEO** (Search Engine Optimization): **Googlebot** no "ve" la pantalla visualmente; lee el **árbol DOM**. Si usas etiquetas **semánticas** (`<article>`, `<nav>`, `<h1>`), Google entiende la **jerarquía** de tu información y te **posiciona mejor** en las búsquedas.
3. **Accesibilidad**: Los **lectores de pantalla** para personas ciegas utilizan las etiquetas semánticas para crear un **"mapa"** auditivo de la web.

Analogía (El Libro): Un sitio web hecho solo con `<div>` es como un libro impreso sin capítulos, sin párrafos, sin negritas y sin índice; solo un **bloque gigante** de texto. El **HTML semántico** añade los títulos, los índices y las separaciones lógicas.

Etiquetas Semánticas Clave 2026:

- `<header>` y `<footer>`: Zonas **superior** e **inferior** de la página o de un artículo.
- `<nav>`: Solo para la **navegación principal** del sitio.
- `<main>`: Envuelve el **contenido central**. Regla de oro: **Solo un `<main>` por página**.
- `<article>`: Contenido **autoconclusivo** que tendría sentido por sí solo en otra web (ej. una tarjeta de producto, un post de blog).
- `<section>`: **Agrupador temático** (ej. una sección de "Testimonios" o "Contacto"). Suele requerir su propio **encabezado** (`<h2>`).
- `<aside>`: **Contenido complementario** (barras laterales, publicidad).
- `<search>`: (Nueva) Contenedor semántico moderno para los **formularios de búsqueda**, vital para **SEO** y **accesibilidad**.

3.4. Accesibilidad (A11y) y Estándar WCAG 2.2

La **Accesibilidad (A11y)** garantiza que tu web sea utilizable por personas con **discapacidades** motrices, visuales o cognitivas. Las pautas internacionales actuales son las **WCAG 2.2**.

1. Estructura de Encabezados (Headings):

Los **encabezados** (`<h1>` a `<h6>`) deben seguir un **orden estricto**. Nunca te saltes un nivel (no pases de un `<h1>` a un `<h3>` directamente). Solo debe haber un `<h1>` que describa el **propósito general de la página**.

2. Navegación por Teclado y Foco:

Muchos usuarios navegan únicamente con la tecla **Tab**.

- Los **elementos interactivos** (`<button>`, `<a>`, `<input>`) reciben el **"foco"** automáticamente.
- Nunca escondas el **anillo de foco** del navegador sin proporcionar uno personalizado (`outline: none` sin un reemplazo visual es un **fallo de A11y gravísimo**).
- Nunca uses un `<div>` con un evento JavaScript `onclick` simulando ser un botón. Los **usuarios de teclado no podrán acceder** a él.

3. Formularios e Imágenes:

- Todo `<input>` debe tener un `<label>`. Se vinculan usando `for` en el label y el `id` en el input.
- Toda `<img>` requiere el atributo `alt`. Si la imagen aporta información, **describela** (`alt="Gráfico de ventas"`). Si es puramente **decorativa**, déjalo **vacío** (`alt=""`) para que el lector de pantalla la ignore en lugar de leer el nombre del archivo.

4. Roles y ARIA (Lo esencial):

ARIA (Accessible Rich Internet Applications) es un conjunto de atributos para añadir semántica donde el HTML nativo se queda corto (muy usado en Angular o React al crear componentes complejos).

- `aria-label="Cerrar ventana"`: Da **nombre** a botones que solo tienen íconos **sin texto**.
- `aria-hidden="true"`: **Ocult**a elementos visuales **irrelevantes** para los lectores de pantalla (como íconos SVG).
- `aria-expanded="true/false"`: Indica el **estado** de un **menú desplegable** o acordeón.

3.5. Plantilla Base (Boilerplate) Profesional

Este es el esqueleto ideal para un documento en 2026. ¡Guárdalo como referencia!

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- SEO Básico y Título -->
  <title>TaskFlow | Gestión de Tareas</title>
  <meta name="description" content="Organiza tu productividad diaria con TaskFlow.">

  <!-- Importando el CSS (Director llama a los músicos) -->
  <link rel="stylesheet" href="/styles/main.css">
</head>
<body>

  <!-- Cabecera y Navegación Principal -->
  <header>
    <h1>TaskFlow 2026</h1>
    <nav aria-label="Navegación principal">
      <ul>
        <li><a href="#inicio">Inicio</a></li>
        <li><a href="#caracteristicas">Características</a></li>
      </ul>
    </nav>
  </header>

  <!-- Contenido Único de la Página -->
  <main>
    <!-- Funcionalidad de Búsqueda Semántica -->
    <search>
      <form action="/buscar" method="GET">
        <label for="buscador">Buscar tarea:</label>
        <input type="search" id="buscador" name="q" placeholder="Ej. Comprar pan">
        <!-- Botón accesible con icono (aria-label provee el contexto oculto) -->
        <button type="submit" aria-label="Ejecutar búsqueda">
          <svg aria-hidden="true" viewBox="0 0 20 20"><!-- icono --></svg>
        </button>
      </form>
    </search>

    <!-- Agrupador Temático -->
    <section id="caracteristicas">
      <h2>Nuestras Características</h2>
      <article>
        <h3>Sincronización en la nube</h3>
        <p>Accede a tus datos desde cualquier dispositivo.</p>
        <!-- Imagen decorativa ignorada por el lector de pantalla -->
        
      </article>
    </section>
  </main>

  <footer>
    <p>© 2026 UTN. Todos los derechos reservados.</p>
  </footer>

  <!-- JS cargado de forma optimizada (defer) -->
  <script src="/scripts/app.js" defer></script>
</body>
</html>
```

Errores Comunes

1. **Rutas Rotas:** Escribir `` (Ruta **absoluta local**). Esto funcionará en tu **PC**, pero se **romperá** en cuanto subas el proyecto a un **servidor** o a **GitHub**. Solución: Usar **rutas relativas** (`./imagen.jpg`).
2. **Div-itis Crónica:** Construir toda la **página** usando `<div class="nav">` o `<div class="footer">`. Solución: Reemplazarlos por `<nav>` y `<footer>`.
3. **Múltiples `<h1>`:** Creer que `<h1>` significa "**texto grande**" y usar varios en la misma **página** para **estilizar** el texto. Solución: Usar el `<h1>` solo para el **título** del documento y controlar el **tamaño** de otras letras con **CSS**.

4. CSS Esencial: Layouts Modernos y Tailwind

Para dominar la web (y herramientas como Tailwind o Angular), primero debes entender la base de la presentación: CSS (Cascading Style Sheets).

4.1 ¿Qué es CSS y cómo funciona en la Web?

CSS es un **lenguaje de diseño** utilizado para describir la **presentación** de un documento **HTML**. Funciona separando estrictamente el **contenido (HTML)** del **diseño (CSS)**.

Teoría del Renderizado (El CSSOM): Cuando el navegador lee tu **HTML**, construye el árbol del **DOM**. Simultáneamente, lee tu archivo **CSS** y construye el **CSSOM (CSS Object Model)**. Luego, fusiona ambos árboles para formar el "**Render Tree**", que es lo que finalmente se pinta (píxel por píxel) en tu pantalla.

La Cascada y la Especificidad: La "**C**" de **CSS** significa **Cascada**. Si dos **reglas CSS** intentan darle estilo al mismo elemento, el navegador decide cuál gana basándose en un sistema de **jerarquías**:

1. **Especificidad** (El sistema de puntos): El navegador calcula un "**peso**" para cada regla.
 - Etiquetas en línea (`style="..."`): Tienen el **peso máximo (1000 puntos)**.
 - **Selectores de ID (#mi-id): 100 puntos.**

- **Selectores de Clase, pseudoclasses o atributos (mi-clase): 10 puntos.**
 - **Selectores de Etiqueta o pseudoelementos (p, div): 1 punto.**
 - Ejemplo: Una **clase** gana sobre una **etiqueta**, pero un **ID** le gana a infinitas clases.
2. **Ubicación del CSS** (Archivos vs Etiquetas **<style>**): ¿Qué vale más, un archivo **style.css externo** o una etiqueta **<style>** dentro del **HTML**?
- Tienen la **misma jerarquía** a nivel de **especificidad**. El desempate lo define el **orden de aparición** en el documento **HTML**.
 - Si enlazas **<link href="style.css">** y después pones una etiqueta **<style>**, los **estilos internos** sobrescribirán a los del archivo **externo**.
 - Sin embargo, los **estilos en línea** directamente en la etiqueta HTML (**<div style="color: red;">**) siempre ganarán sobre ambos, ya que tienen la **especificidad máxima**.
3. **Orden de aparición**: Si dos reglas tienen exactamente la **misma especificidad** y están en el mismo nivel, la **regla que se lea última (más abajo en el código)** será la que se aplique.
4. La regla nuclear (**!important**): Si añades **!important** al final de un valor (ej. **color: red !important;**), esta regla **romperá toda la cascada** y **ganará sobre cualquier otra cosa** (incluso sobre los **estilos en línea**). Su uso debe **evitarse** a menos que sea estrictamente necesario, ya que hace que el código sea muy **difícil de mantener**.

4.2 El Modelo de Caja (Box Model)

Todo elemento en **HTML** es renderizado como una **caja rectangular**. Comprender las partes de esta caja es **vital** para no "romper" los **diseños**.

- **Content (Contenido)**: El área donde reside el **texto** o la **imagen**.
- **Padding (Relleno)**: El **espacio transparente adentro** de la caja (empuja el contenido hacia adentro).
- **Border (Borde)**: La **línea** que rodea al padding y al contenido.
- **Margin (Margen)**: El **espacio transparente afuera** de la caja (**separa** esta caja de otras).

Analogía: Piensa en un cuadro en la pared. La pintura es el **Content**, el paspartú blanco alrededor de la pintura es el **Padding**, el marco de madera es el **Border**, y la distancia entre ese cuadro y el cuadro de al lado es el **Margin**.

El salvavidas CSS (box-sizing):

Por defecto, si le das a una caja **width**: 100px, un **padding**: 10px y un **border**: 2px, el **ancho real** será **124px** ($100 + 10 + 10 + 2 + 2$). Esto vuelve las **matemáticas** de las columnas un **infierno**.

Regla de oro estándar de la industria: Aplicar **box-sizing: border-box** a todo, lo cual fuerza a que el **width** englobe el **padding** y el **border**.

```
*, ::before, ::after {  
  box-sizing: border-box;  
}
```

4.2 Layouts Modernos: Flexbox

Flexbox (**display: flex**) es un **motor de diseño** creado específicamente para organizar **elementos** en una **sola dimensión** a la vez (ya sea en una fila o en una columna).

Teoría (Los Ejes):

Flexbox trabaja sobre **dos ejes**:

- **Main Axis (Eje Principal):** Determinado por **flex-direction**. Si es **row** (por defecto), es **horizontal** (de izquierda a derecha).
- **Cross Axis (Eje Transversal):** **Perpendicular** al principal. Si **flex-direction** es **row**, este eje es **vertical**.

Analogía: Imagina las **perlas** de un **collar**. Puedes deslizarlas a lo largo del **hilo**, juntarlas al **centro**, o separarlas **equitativamente**.

- **justify-content:** Alinea las "perlas" a lo largo del hilo (Main Axis). Ej: center, space-between.
- **align-items:** Ajusta cómo se alinean las perlas respecto al grosor del collar (Cross Axis). Ej: center, flex-start.

```
.contenedor-flex {
  display: flex;
  flex-direction: row;      /* Fila horizontal */
  justify-content: space-between; /* Empuja los elementos a los extremos */
  align-items: center;      /* Los centra verticalmente */
  flex-wrap: wrap;          /* Si no caben, pasan a la siguiente línea */
  gap: 16px;                /* Espacio consistente entre elementos */
}
```

4.4 Layouts Modernos: CSS Grid

A diferencia de **Flexbox**, **CSS Grid** (**display: grid**) fue creado para controlar diseños bidimensionales (**filas Y columnas** al mismo tiempo).

Analogía: Es como una hoja de cálculo (**Excel**) o un tablero de ajedrez, donde defines el número de celdas horizontales y verticales, y luego decides cuántas celdas ocupa cada elemento.

- Unidad fr (Fraction): Una unidad mágica de Grid que representa "una fracción del espacio libre disponible".

```
.contenedor-grid {
  display: grid;
  /* Crea 3 columnas: la 1ra y 3ra de 1 fracción, la del medio el doble de ancha (2fr) */
  grid-template-columns: 1fr 2fr 1fr;
  gap: 20px; /* Separación entre filas y columnas */
}

/* Un elemento hijo puede expandirse por varias celdas */
.hijo-destacado {
  grid-column: span 3; /* Ocupa las 3 columnas enteras */
}
```

4.5 Subgrid: La Revolución

Un problema histórico de Grid era la alineación de los contenidos internos de varias tarjetas distintas. Por ejemplo, si el título de una de las tres tarjetas de productos ocupaba dos líneas, los botones de "Comprar" quedaban desalineados.

Subgrid **soluciona** esto al permitir que los **elementos hijos hereden** la **cuadrícula** de su **padre**.

```
/* El padre tiene el grid principal */
.padre-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}

/* Cada tarjeta también es un grid, pero sus filas heredan del contenedor exterior */
.tarjeta-hija {
  display: grid;
  grid-template-rows: subgrid;
  grid-row: span 3; /* Título, párrafo, botón */
}

/* ¡Ahora los botones de "Comprar" de todas las tarjetas estarán siempre en la misma
línea! */
```

4.6 Tailwind CSS: El Framework Definitivo

Para 2026, escribir **CSS puro** en archivos separados es poco común en grandes equipos. El estándar es **Tailwind CSS**.

¿Es una Librería o un Framework?

Tailwind es un **Framework completo**. Una librería (como Bootstrap tradicional) simplemente te da componentes pre-diseñados (.btn-primary) que instalas y usas. Tailwind dicta una **arquitectura y metodología de trabajo (Utility-First)** e incluye su propio **motor de compilación**.

El Paradigma Utility-First:

En lugar de crear una clase semántica (.tarjeta-perfil), Tailwind te da **clases de bajo nivel (utilidades)** que aplican una sola regla CSS (ej. flex, p-4 (padding), text-center, text-blue-500). Diseñas construyendo directamente en el **HTML**.

Puntos Fuertes:

1. **Velocidad de desarrollo:** No cambias de contexto entre HTML y CSS, ni inventas nombres de clases.
2. **Diseño Responsivo integrado:** Facilita adaptar la web a móviles usando prefijos como **md:** o **lg:** (ej. `<div class="w-full md:w-1/2">`).
3. **Mantenibilidad absoluta:** Al eliminar código HTML, su estilo desaparece. Jamás tendrás un archivo CSS obeso lleno de **código muerto (CSS huérfano)**.

Optimización y Rendimiento (**Motor JIT y Purge**):

Una duda frecuente: "Si Tailwind tiene decenas de miles de clases, ¿mi proyecto pesará muchísimo?"

NO. Tailwind usa un motor **JIT (Just-In-Time)**. Cuando corres tu proyecto localmente, Tailwind escanea todos tus archivos `.html` y `.js`, lee exactamente qué clases usaste (ej. `text-red-500`) y genera un **archivo CSS final** que contiene **ÚNICAMENTE** esas clases exactas. El archivo CSS de **producción** resultante **rara vez supera los 10 KB**.

4.7 Instalación y Configuración Profesional con Vite

Para usar Tailwind **profesionalmente** (y aprovechar su **compilador optimizado**), se instala vía **NPM** junto con **empaquetadores** como **Vite**.

1. Crear el proyecto Vite e instalar Tailwind:

- Crear proyecto web con **Vite (vanilla JS)**
`npm create vite@latest mi-proyecto -- --template vanilla`
`cd mi-proyecto`
- Instalar **Tailwind** y sus **conectores (PostCSS, autoprefixer)** como **dependencias de desarrollo**
`npm install -D tailwindcss postcss autoprefixer`
- Generar **archivo de configuración (tailwind.config.js)**
`npx tailwindcss init -p`

2. Configurar las **rutas de las plantillas (Crucial para la optimización)**:

El archivo `tailwind.config.js` es donde le indicas al **compilador** qué archivos debe escanear para que la **purga (Tree-shaking)** funcione.

```
// tailwind.config.js
/** @type {import('tailwindcss').Config} */
export default {
  // Define dónde Tailwind buscará clases usadas
  content: [
    "./index.html",
    "./src/**/*.js,ts,jsx,tsx",
  ],
  theme: {
    extend: {}, // Aquí puedes agregar tus colores personalizados
  },
  plugins: [],
}
```

1. Inyectar Tailwind en tu CSS principal (style.css):

Borra el contenido predeterminado y coloca estas tres capas:

```
/* style.css */
@tailwind base;      /* Reinicia estilos nativos del navegador */
@tailwind components; /* Para componentes personalizados si existen */
@tailwind utilities; /* Las clases utilitarias de Tailwind */
```

2. Levantar el entorno:

npm run dev

Ejemplo de Componente Construido con Tailwind CSS:

```
<article class="p-6 bg-white rounded-2xl shadow-xl hover:shadow-2xl transition-all
md:flex md:items-center md:space-x-6">
  <div class="flex-shrink-0">
    <div class="h-16 w-16 bg-blue-100 text-blue-600 rounded-full flex items-center
justify-center font-bold text-2xl">
      PRO
    </div>
  </div>
  <div class="mt-4 md:mt-0">
    <h3 class="text-xl font-bold text-slate-800">Suscripción Premium</h3>
    <p class="mt-2 text-slate-600 text-sm md:text-base">
      Accede a todas las herramientas. El diseño responde si estás en móvil (`mt-4`) o
      escritorio (`md:mt-0`).
    </p>
    <button class="mt-4 bg-blue-600 text-white font-semibold py-2 px-6 rounded-lg
hover:bg-blue-700 focus:ring-4 focus:ring-blue-300 transition-colors">
      Comprar ahora
    </button>
  </div>
</article>
```

4.8 Documentación de Referencia (Tailwind Docs):

Para dominar las clases disponibles y leer sobre arquitectura de componentes, visita:

- [Core Concepts: Utility-First \(Docs Oficiales\)](#)
- [Instalación con Vite](#)

5. JavaScript: Fundamentos, Arquitectura y Calidad de Código

JavaScript es un lenguaje **multiparadigma**, **dinámico** e **interpretado** (con **compilación JIT**). Para dominar **frameworks modernos** como **Angular** o **React**, no basta con saber escribir condicionales o bucles; debes comprender su **sintaxis**, cómo gestiona la **memoria**, sus **flujos asíncronos** y los **patrones de desarrollo profesional**.

5.1 Fundamentos del Lenguaje (Variables, Tipos y Operadores)

Variables, Ámbito (Scope) y Hoisting

- **const** y **let** (ES6+): Tienen **ámbito de bloque (Block Scope)**. Solo existen dentro de las llaves `{ }` donde fueron declaradas. Regla de oro: Usa siempre **const** por defecto. Si sabes que el valor cambiará (ej. en un contador `for`), usa **let**.
- **var** (Obsoleto): Tiene **ámbito de función (Function Scope)** y sufre de un comportamiento llamado **Hoisting** complejo, donde la declaración se "eleva" al inicio pero no su valor, causando **bugs** impredecibles (`undefined`). No uses **var** en código moderno.

Tipos de Datos

- **Tipos Primitivos**: **String**, **Number** (no hay diferencia entre enteros y flotantes), **Boolean**, **Undefined**, **Null**, **BigInt**, y **Symbol**.
- **Estructuras de Datos**: **Objetos** (`{ }`) y **Arrays** (`[ ]`).

Truthy y Falsy

En **JS**, cualquier valor puede ser evaluado como un Booleano en un contexto lógico (ej. dentro de un `if`).

- **Falsy** (falsos): **0**, **""** (string vacío), **null**, **undefined**, **NaN**, **false**.
- **Truthy** (verdaderos): Absolutamente **todo lo demás** (incluyendo `[ ]` arrays vacíos, `{ }` objetos vacíos y `"0"` strings con texto cero).

Operadores y Características Modernas Esenciales

2026 exige dominar estas herramientas de sintaxis avanzada:

```
// 1. Template Literals (Backticks): Interpolación limpia sin concatenar con '+'
const nombre = "Juan";
const msg = `Hola ${nombre}, bienvenido.`;

// 2. Destructuración (Destructuring): Extraer valores de objetos o arrays directamente
const usuario = { id: 1, email: "test@test.com" };
const { id, email } = usuario;

// 3. Operador Spread/Rest (...): Expande elementos o los agrupa
const nums = [1, 2, 3];
const copiaYMas = [...nums, 4, 5]; // [1, 2, 3, 4, 5]

// 4. Optional Chaining (?.) y Nullish Coalescing (??)
// Evita errores crasheantes como "Cannot read property of undefined"
const ciudad = usuario.direccion?.ciudad; // Devuelve undefined si 'direccion' no existe
const nombreFinal = usuario.nombre ?? "Anónimo"; // "Anónimo" solo si 'nombre' es null o undefined
```

5.2 Funciones, Contexto y Closures

En JS, las funciones son "Ciudadanos de Primera Clase" (*First-Class Citizens*). Son valores que puedes asignar a variables, pasar como argumentos (Callbacks) o retornar.

1. **Funciones Tradicionales:** Crean su propio contexto **this** dependiendo de cómo son **invocadas**.

```
// Declarada (Sufre Hoisting, puedes llamarla antes de definirla)
function sumar(a, b = 0) {
  return a + b;
}
```

2. **Arrow Functions (Funciones Flecha):** No tienen su propio **this**. Heredan el **this** del **contexto exacto** donde fueron escritas (**Lexical Scope**).

```
const usuario = {
  nombre: "Ana",

  // ERROR: El setTimeout invoca esta función tradicional en el contexto global (Window)
  saludarMal: function() {
    setTimeout(function() { console.log("Hola " + this.nombre); }, 1000); // "Hola undefined"
  },

  // SOLUCIÓN: La Arrow Function "recuerda" que fue creada dentro de 'usuario'
  saludarBien: function() {
    setTimeout(() => { console.log("Hola " + this.nombre); }, 1000); // "Hola Ana"
  }
};
```

Closures (Clausuras): Memoria Fotográfica

Un Closure es la capacidad de una función interna de "recordar" y acceder a las variables de la función externa que la envolvió, incluso después de que la función externa haya terminado de ejecutarse.

El Garbage Collector (GC) (recolector de basura) normalmente destruye las variables de una función cuando esta termina. Pero si esa función retorna una hija que aún necesita las variables, el GC las mantiene vivas en un "Ámbito Léxico".

```
// Ejemplo Práctico: El Contador Privado
function crearContador() {
  let cuenta = 0; // Variable "privada"

  return function() {
    cuenta++; // Accede a 'cuenta', que sigue viva en la "mochila" del closure
    return cuenta;
  };
}

const miContador = crearContador();
console.log(miContador()); // 1
console.log(miContador()); // 2
// Nadie desde afuera puede hacer `cuenta = 999`. Está protegida.
```

5.3 El Modelo de Ejecución y el Event Loop

A diferencia de lenguajes como Java o C++, **JavaScript** en el navegador es **Single-Threaded** (de un solo hilo). Esto significa que solo puede procesar una instrucción a la vez en su **Call Stack** (Pila de llamadas).

Si una tarea toma 5 segundos, toda la pestaña del navegador se **congelará**. Para evitar esto, JS delega las tareas lentas a las **Web APIs** del navegador.

¿Cómo funciona la **asincronía**? (El **Event Loop**)

Existen dos **colas** principales para recibir tareas terminadas:

1. **Macrotask Queue** (Cola de Tareas): Aquí van los *callbacks* de `setTimeout` o `setInterval`. Tienen **prioridad normal**.
2. **Microtask Queue** (Cola de Microtareas): Aquí van las **Promesas** (`.then()`, `.catch()`) y la API `queueMicrotask`. Tienen **Prioridad Absoluta (VIP)**.

Analogía (**El Restaurante**): El **Call Stack** es el único mozo del restaurante. Las **Web APIs** son la cocina. La **Macrotask Queue** es la fila normal de clientes. La **Microtask Queue** es la fila **VIP**. El **Event Loop** es el gerente de la puerta: siempre vaciará completamente la fila **VIP** antes de dejar pasar a un solo cliente de la fila normal.

```
// Ejemplo Práctico: El orden de ejecución (Pregunta de entrevista)
console.log('1. Código Síncrono (Call Stack)');

setTimeout(() => {
  console.log('4. Macrotask (Timeout de 0ms)');
}, 0);

queueMicrotask(() => {
  console.log('3. Microtask (Función VIP encolada)');
});

Promise.resolve().then(() => {
  console.log('2. Microtask (Promesa resuelta)');
});

// SALIDA ESPERADA: 1 -> 2 -> 3 -> 4
// Aunque setTimeout tenía 0ms, fue desplazado por las microtareas.
```

5.4 Memoria, Inmutabilidad y Copias

Comprender cómo JS maneja la **memoria** te ahorrará horas de depuración.

- **Tipos Primitivos (Pasados por Valor)**: Si haces `let a = 10; let b = a;`, se crea una **copia exacta y desconectada**. Modificar **b** no altera **a**.
- **Tipos de Referencia (Objetos y Arrays)**: Si haces `const obj2 = obj1;`, solo copias la **dirección de memoria**. **Mutar** uno, **muta** el otro irremediabilmente.

En Frontend moderno (React/Angular) promovemos la **Inmutabilidad**: **Nunca modificamos el original**, creamos una **copia limpia**.

1. Shallow Copy (Copia Superficial) con Spread `...`:

Copia el primer nivel, pero los objetos anidados dentro seguirán compartiendo la referencia.


```
const user = { nombre: "Ana", config: { tema: "dark" } };
const userCopia = { ...user, nombre: "Ana María" };

userCopia.config.tema = "light";
console.log(user.config.tema); // ¡"light"! (El anidado mutó el original)
```

2. Deep Copy (Copia Profunda) con structuredClone

Es la API nativa moderna para clonar objetos profundamente, rompiendo toda referencia.

```
const original = { tema: "dark", config: { rol: "admin" } };
const clonProfundo = structuredClone(original);

clonProfundo.config.rol = "invitado";
console.log(original.config.rol); // "admin" (¡El original está a salvo!)
```

5.5 Objetos, Prototipos y Clases (OOP en JS)

JS utiliza **Herencia Prototípica**. Todo **objeto** en JS está enlazado a otro objeto "oculto" llamado su **prototipo**. Si se solicita una **propiedad** que el objeto no posee, JS la buscará en su prototipo, formando una **cadena** de prototipos.

La sintaxis class:

Introducida en ES6, es "azúcar sintáctico" (*syntactic sugar*) para facilitar la comprensión y el uso de la herencia prototípica por parte de desarrolladores familiarizados con modelos de herencia tradicionales.

```
class Usuario {
  constructor(nombre) {
    this.nombre = nombre;
  }
  // Este método se añade al Prototype para ahorrar memoria compartida
  saludar() {
    return `Hola, ${this.nombre}`;
  }
}

// 'new' crea un objeto {}, enlaza su 'this' y su prototipo a la Clase
const admin = new Usuario('Laura');
```

Nota: En Angular, la POO y las clases (decoradas con @Component) siguen siendo el estándar absoluto de la arquitectura.

5.6 Módulos ES (Import / Export)

Históricamente, los scripts en HTML chocaban entre sí por usar el mismo espacio global. Los ES Modules solucionan esto: cada archivo es un universo aislado. Para compartir código, debes exportarlo explícitamente.

```
// --- utilidades.js ---  
// Exportación Nombrada (puedes tener varias)  
export const PI = 3.1416;  
export function sumar(a, b) { return a + b; }  
  
// Exportación por Defecto (solo una por archivo)  
export default class Calculadora { /*...*/ }  
  
// --- app.js ---  
// Importación combinada  
import Calculadora, { PI, sumar } from './utilidades.js';
```

5.7 Errores y Manejo de Excepciones

Un buen desarrollador asume que la red fallará. JS nos da try/catch/finally para interceptar explosiones.

```
async function obtenerPerfil(userId) {  
  try {  
    // fetch salta al 'catch' SOLO si la red se cae físicamente  
    const response = await fetch(`https://api.test.com/users/${userId}`);  
  
    // Debemos verificar manualmente los errores HTTP (404, 500) y lanzar error  
    if (!response.ok) throw new Error(`Error HTTP: ${response.status}`);  
  
    return await response.json();  
  } catch (error) {  
    console.error("Fallo al obtener perfil:", error.message);  
  } finally {  
    // Se ejecuta SIEMPRE. Ideal para apagar *spinners* (loaders) de carga.  
    console.log("Petición finalizada.");  
  }  
}
```

5.8 Buenas Prácticas de Calidad de Código (Clean Code)

1. `===` vs `==`: **SIEMPRE** usa la **igualdad estricta** (`===`). La igualdad débil (`==`) **convierte tipos automáticamente**, causando **bugs** absurdos.
2. **Naming (Nombres claros)**: Evita **abreviaturas** como `usr` o `func1`. Usa **nombres descriptivos** como `usuarioData` o `calcularImpuesto()`.
3. Evitar **Variables Globales**: El objeto `window` debe permanecer **limpio**.
4. **Funciones Puras**: Una función pura **siempre devuelve el mismo resultado** dados los mismos parámetros y **no tiene efectos secundarios** (no lee globales, no modifica el DOM). **Separa** la lógica de **negocio/matemática** de la **interfaz visual**.

5.9 Seguridad Básica: Prevención de XSS

XSS (Cross-Site Scripting) es una vulnerabilidad crítica donde un atacante inyecta JS malicioso en el navegador de otro usuario.

La regla cardinal: Nunca confíes en el texto que ingresa un usuario.

Si usas la propiedad `innerHTML`, el navegador ejecutará cualquier `<script>` oculto.

```
const comentarioUsuario = "<img src='x' onerror='alert(\"Hackeado\")'>";

const div = document.querySelector('#mi-contenedor');

// ✗ PELIGROSO (Vulnerabilidad XSS)
div.innerHTML = comentarioUsuario;

// ✔ SEGURO (El navegador lo renderiza como texto literar)
div.textContent = comentarioUsuario;
```

5.10 Errores Típicos

- Error 1: **Mutación accidental de Estado**. Evitar realizar operaciones como `.sort()` o `.push()` directamente sobre un **Array original**. Esta práctica **rompe la reactividad** en *frameworks* modernos. **Solución**: **Clonar** primero el *array* con la sintaxis `[...datos].sort()`.
- Error 2: **Falsos positivos asíncronos**. Intentar usar una variable que espera un `fetch` sin utilizar `await`. Esto resultará en la impresión en pantalla de un **Promise { <pending> }** en lugar de los **datos reales**.

- Error 3: **Olvidar exportar módulos**. Es un error común encontrar un **import no encontrado** a pesar de tener una lógica perfecta, simplemente por **olvidar** poner la palabra clave **export** antes de la función o clase.
- Error 4: **Bloquear el Call Stack**. Ejecutar un **bucle for** con una **gran cantidad de iteraciones** (ej. 10 millones) de forma **sincrónica**, lo que **congela** por completo el **navegador** del cliente.

6. DOM (Document Object Model): Estructura, Render y Rendimiento

El Document Object Model (DOM) es la interfaz de programación que representa la página web (HTML o XML). Convierte el código HTML estático en una jerarquía de nodos "vivos" a los que JavaScript puede acceder para leer o modificar la interfaz en tiempo real.

6.1. El Ciclo de Renderizado del Navegador (Critical Rendering Path)

Para manipular el **DOM** (Document Object Model) como un ingeniero de software (y entender por qué existen **frameworks** como React o Angular), primero debes entender cómo el **navegador** dibuja la pantalla. No es mágico; es una cadena de montaje intensiva y secuencial:

1. **Parsing (Análisis)**: El navegador lee el archivo **.html** y construye el **DOM Tree (Árbol de etiquetas)**.
2. **CSSOM**: Paralelamente, procesa el **CSS** y construye el **CSS Object Model (Árbol de estilos)**.
3. **Render Tree**: El navegador **fusiona** el **DOM** y el **CSSOM**. Nodos invisibles (ej. los que tienen **display: none** o la etiqueta **<head>**) se **descartan** para **ahorrar memoria**.
4. **Layout (Reflow)**: El motor calcula la **matemática exacta** de cada nodo: qué **tamaño** tiene, en qué **coordenadas (x, y)** va, y cómo afecta a sus vecinos.
5. **Paint (Pintado)**: Rellena los **píxeles reales** de cada caja (color de fondo, bordes, texto).

6. **Compositing**: Si hay **capas superpuestas** (índices **z**, animaciones **3D** o **transformaciones**), la **tarjeta gráfica (GPU)** compone la imagen final.

Analogía (La Obra de Construcción): El **DOM** es poner los **ladrillos**. El **CSSOM** es comprar la **pintura y ventanas**. El **Render Tree** es el **plano final autorizado**. **Layout** es calcular los **metros cuadrados**. **Paint** es pintar la **pared**, y **Compositing** es tomarle una **foto** a la casa terminada. Si mueves una pared con **JS** (**JavaScript**), obligas al equipo a ejecutar **Layout**, **Paint** y **Compositing** de nuevo.

Estructura del Árbol:



6.2. Selección y Navegación Moderna (Traversing)

Atrás quedaron métodos viejos como `getElementById`. En 2026 estandarizamos usando selectores CSS precisos.

```
// Selección Directa
const btnLogin = document.querySelector('.btn-primary');
const tarjetas = document.querySelectorAll('article.card'); // Devuelve una NodeList estática

// Traversing (Navegación relativa)
const contenedorPadre = btnLogin.parentElement;
const primerHijo = contenedorPadre.firstElementChild;
const hermano = btnLogin.nextElementSibling;
```

APIs Modernas Esenciales: `closest` y `matches`

En lugar de saltar ciegamente con `parentElement.parentElement`, usamos selectores inteligentes.

```
const boton = document.querySelector('.btn-delete');

// matches() verifica si el elemento cumple una regla CSS (devuelve true/false)
if (boton.matches('.is-disabled')) return;

// closest() busca hacia arriba el ancestro MÁS CERCANO que cumpla la regla.
// ¡Es la herramienta definitiva para la delegación de eventos!
const filaTabla = boton.closest('tr.table-row');
filaTabla.style.backgroundColor = 'red';
```


6.3. Modificación, Atributos y APIs 2026

Manipular el **DOM** hoy es mucho más **limpio** que hace 10 años. Tenemos **APIs dedicadas** para las **tareas comunes**.

Clases y Atributos Dataset:

```
const el = document.querySelector('#mi-elemento');

// classList (Mucho mejor que sobrescribir con className)
el.classList.add('fade-in');
el.classList.remove('oculto');
el.classList.toggle('activo'); // Quita si existe, agrega si no existe.

// dataset (Acceso a atributos de HTML que empiezan con data-*)
// HTML: <div data-user-id="123" data-role="admin">
console.log(el.dataset.userId); // "123"
el.dataset.role = "superadmin"; // Modifica el HTML en tiempo real
```

Atributos Booleanos (toggleAttribute):

Ideal para estados de UI como deshabilitar botones o campos.

```
const inputClave = document.querySelector('#password');
// Activa o desactiva 'disabled' según un booleano
inputClave.toggleAttribute('disabled', modoLectura);
```

6.4. Rendimiento Extremo: El Terror del "Layout Thrashing"

Cada vez que JavaScript lee una **dimensión del DOM** (ej. `offsetHeight`), el navegador debe asegurarse de que la pantalla esté **actualizada**. Si intercalas **lecturas con escrituras** en un bucle, fuerzas al navegador a realizar el paso de **Layout** una y otra vez de forma sincrónica. A esto se le llama **Layout Thrashing** (**Apalizamiento del diseño**), y destruye los **60 FPS** (**Fotogramas por Segundo**).

```
const cajas = document.querySelectorAll('.caja');
const contenedor = document.querySelector('#contenedor');

// ❌ PÉSIMO RENDIMIENTO (Layout Thrashing)
cajas.forEach(caja => {
  // Lee (fuerza layout) y luego escribe (invalida layout). ¡Repetido 100 veces!
  const anchoDeseado = contenedor.offsetWidth;
  caja.style.width = anchoDeseado + 'px';
});

// ✅ EXCELENTE RENDIMIENTO (Batch Updates)
// Primero TODAS las lecturas...
const anchoDeseado = contenedor.offsetWidth;

// ...Y luego TODAS las escrituras (El navegador hará un solo Layout al final)
cajas.forEach(caja => {
  caja.style.width = anchoDeseado + 'px';
});
```


Sincronización visual con `requestAnimationFrame`:

Si vas a hacer múltiples escrituras complejas (ej. animaciones manuales), dile al navegador que las aplique justo antes del próximo pintado de pantalla.

```
requestAnimationFrame(() => {  
  cajaAnimada.style.transform = `translateX(500px)`;  
});
```

6.5. Creación Eficiente y Renderizado de Listas

Insertar **nodos** uno por uno en el **DOM real** dispara **cálculos innecesarios**. Usamos **DocumentFragment** (una "bolsa" en **memoria** que no afecta la pantalla) para **agrupar inserciones**.

Ejemplo: **Actualización Parcial Eficiente** (`replaceChildren`)

La **API `replaceChildren`** es la forma más **rápida** y **segura** de vaciar un contenedor y meterle datos nuevos.

```
const lista = document.querySelector('#lista-usuarios');  
const datosNuevos = ['Ana', 'Carlos', 'Pedro'];  
  
// 1. Creamos el fragmento invisible  
const fragmento = document.createDocumentFragment();  
  
// 2. Construimos todos los nodos en la bolsa RAM  
datosNuevos.forEach(nombre => {  
  const li = document.createElement('li');  
  li.textContent = nombre;  
  fragmento.append(li);  
});  
  
// 3. Vaciamos la lista vieja e insertamos el fragmento de un solo golpe  
lista.replaceChildren(fragmento);
```

Inyección pura con `insertAdjacentHTML`:

Si solo quieres agregar un string HTML sin sobrescribir (y sin el costo de `innerHTML +=`), usa esto:

```
// Inserta justo antes de que termine la caja (sin destruir el contenido actual)  
contenedor.insertAdjacentHTML('beforeend', '<p>Nuevo nodo rápido</p>');
```

6.6. Seguridad: `textContent` vs `innerHTML` (Riesgo XSS)

Nunca confíes en el usuario. Si tomas datos de un **input** o de una **base de datos** y los inyectas con **innerHTML**, el **navegador** ejecutará las **etiquetas HTML** (incluyendo `<script>`) que vengan ahí. A esto se le llama **Cross-Site Scripting (XSS)**.

```
// Imagina que esto lo ingresó un hacker en una caja de comentarios
const comentarioMalicioso = "Buen post. <img src='x' onerror='alert(\"Cookie robada!\")'>";

const divComentario = document.createElement('div');

// ❌ VULNERABILIDAD GRAVE: El navegador intentará cargar la imagen, fallará, y ejecutará JS
divComentario.innerHTML = comentarioMalicioso;

// ✅ SEGURO: El navegador escapa los caracteres y lo trata como texto plano
divComentario.textContent = comentarioMalicioso;
```

6.7. El DOM y la Accesibilidad (A11y)

Con JavaScript, a menudo rompemos la **accesibilidad nativa** del navegador (creando modales o menús personalizados). Es nuestra **responsabilidad** repararla gestionando el **Foco**.

- **Manejo del Foco:** Si muestras un *popup* dinámico, debes **forzar** que el **teclado navegue** a él.
- El atributo `tabindex="-1"`: Permite que un elemento **sin foco nativo** (como un `<div>`) reciba **foco exclusivamente mediante JS**, sin interrumpir el **flujo normal del teclado** del usuario.

```
const modal = document.querySelector('#mi-modal');

function abrirModal() {
  modal.classList.remove('oculto');
  // Ajustamos ARIA para lectores de pantalla
  modal.setAttribute('aria-hidden', 'false');
  // Enviamos el foco al modal
  modal.focus();
}
```

6.8. El Ciclo de Vida: `DOMContentLoaded` vs `load`

¿Cuándo debe ejecutarse tu **JavaScript**? Si el **script** corre antes de que el **HTML** exista en el **navegador**, tu `querySelector` devolverá **null**.

```
// ✅ Opción 1: DOMContentLoaded
// Se dispara cuando el HTML (etiquetas) está listo. Las imágenes o iframes aún están
cargando.
// Es el estándar de oro para iniciar tu lógica de UI.
document.addEventListener('DOMContentLoaded', () => {
  iniciarApp();
});

// 🐢 Opción 2: load
// Se dispara en 'window'. Espera a que TODO (imágenes pesadas, CSS, scripts externos) se
descargue.
window.addEventListener('load', () => {
  quitarPantallaDeCarga();
});

(Nota: Si usas <script src="app.js" defer></script>, el navegador simula el
DOMContentLoaded automáticamente por ti).
```

6.9. Errores Comunes (Los "Mata-Proyectos")

1. **Buscar lo inexistente:** Colocar el `<script>` en el `<head>` sin `defer` provoca que el **JavaScript** se ejecute antes de que el `<body>` exista. Síntoma común: `Cannot read properties of null (reading 'addEventListener')`.
2. Ignorar el **XSS** con **innerHTML**: Construir toda la UI mediante **template literals** inyectados con `innerHTML` abre la puerta a **ataques**. Solución recomendada: Usar **createElement** o **sanitizar** el contenido.
3. Muerte por **NodeList**: Intentar usar métodos de `Array` (como `.map()`) directamente sobre el resultado de **querySelectorAll**. Solución: Convertir la lista usando `Array.from(nodos)` o `[...nodos]`.
4. **Zombis en Memoria (Memory Leaks)**: Ejecutar `nodo.remove()` pero mantener el elemento referenciado en un **array global** de JavaScript. Esto evita que el **recolector de basura** lo elimine, causando que el consumo de **RAM** aumente.
5. **Apalizamiento del diseño (Layout Thrashing)**: Leer y escribir estilos (`offsetWidth`, `style.height`) repetidamente en el **mismo bucle**, lo que **congela la experiencia del usuario**.

7. Manejo de Eventos Avanzado y Rendimiento

En la web, los **eventos** son acciones o sucesos (clics, teclas pulsadas, desplazamientos) a los cuales el código **JavaScript** puede **reaccionar**. El navegador implementa el **Patrón Observer**: el **DOM** "emite" una **notificación** y nuestro código registra **oyentes** (**listeners**) que se **ejecutan** cuando esta ocurre.

El **Patrón Observer** es un patrón de diseño de arquitectura de software. Imagina que es como suscribirse a un canal de YouTube (el **Sujeto** o elemento del DOM). No necesitas entrar cada 5 minutos a ver si hay un video nuevo; simplemente te **suscribes** (**addEventListener**) y YouTube te envía una **notificación automática** (**event**) a ti y a todos los demás suscriptores cuando algo ocurre. Esto permite que el código sea **eficiente** y **no bloquee** el navegador preguntando constantemente "¿ya hicieron clic?".

A nivel de ingeniería, un **mal manejo de eventos** es la **causa #1** de aplicaciones **lentas** (**lag**) y **fugas de memoria** (**memory leaks**).

7.1. El Objeto event y Control de Propagación

Cada vez que un listener se ejecuta, recibe un objeto que describe el suceso.

```
boton.addEventListener('click', (event) => {  
  // ¿Dónde ocurrió la acción vs dónde está atado este código?  
  console.log("Clic en:", event.target); // Puede ser un ícono <svg> dentro del  
  botón  
  console.log("Listener en:", event.currentTarget); // Es el <button> que tiene el  
  addEventListener  
  
  // Evitar que un enlace cambie de página o que un formulario recargue la app  
  event.preventDefault();  
});
```

7.2. El Flujo del Evento (Event Flow)

Cuando haces clic en un elemento anidado, el navegador ejecuta tres fases en el flujo de eventos:

1. **Captura** (*Capturing*): El evento **baja** desde *window* -> *document* -> *body* buscando el **elemento objetivo** (*target*).
2. **Target**: El evento **impacta** el **elemento exacto** donde se originó el clic.
3. **Burbujeo** (*Bubbling*): El evento **asciende** de vuelta **notificando** a todos los **elementos padres** (este es el **comportamiento por defecto**).

Control Avanzado del Burbujeo:

- `event.stopPropagation()`: **Evita** que el evento **suba** a los elementos padres en la fase de burbujeo.
- `event.stopImmediatePropagation()`: **Evita** el **burbujeo** y, adicionalmente, **cancela** cualquier otro **listener** del mismo tipo que esté atado al **mismo elemento** donde se ejecutó esta función.

```
const btn = document.querySelector('#btn-critico');

btn.addEventListener('click', (e) => {
  e.stopImmediatePropagation();
  console.log('Se ejecuta esto.');
```

// Este código jamás se ejecutará gracias a stopImmediatePropagation

```
btn.addEventListener('click', () => console.log('Esto queda silenciado'));
```

7.3. Clasificación de Eventos 2026 (Tabla de Referencia)

Categoría	Evento	¿Cuándo usarlo?	Detalle Clave 2026
Pointer (Ratón/Táctil)	pointerdown, pointermove, click	Interacciones de UI.	Unifican clics de mouse, toques de dedo (touch) y lápiz (stylus). Usa la propiedad event.pointerType.
Teclado	keydown, keyup	Atajos, navegación accesible.	Evalúa event.key === 'Escape'. Usa event.repeat para ignorar cuando el usuario deja presionada la tecla.
Formularios	submit, input, change	Validación de datos.	input se dispara tecla por tecla. change espera a que el usuario termine y quite el foco (blur).
Documento	DOMContentLoaded	Inicialización de la App.	Garantiza que el HTML fue leído y el DOM construido, antes de que carguen las imágenes.
Portapapeles	copy, paste	Modificar o limpiar el texto que el usuario copia/pega.	Manipula event.clipboardData para añadir marca de agua al texto copiado.
Drag & Drop	dragstart, dragover, drop	Arrastrar elementos (Trello/Jira).	Requiere event.preventDefault() en el dragover para permitir que el elemento sea soltado (drop).

7.4. Pointer Events y Accesibilidad por Teclado

Pointer Capture (Arrastrar y Soltar Avanzado):

Si el usuario hace **clic y arrastra** un *slider* rápido, el **puntero** puede **salirse** del **área** del elemento, interrumpiendo el evento. `setPointerCapture` "secuestra" el puntero para que los **eventos** sigan enviándose al **elemento original** sin importar adónde vaya el *mouse*.

```
slider.addEventListener('pointerdown', (e) => {  
  slider.setPointerCapture(e.pointerId); // El slider "atrapa" el mouse  
});
```

Accesibilidad Total (Teclado):

Todo lo que se puede hacer con el mouse, debe poder hacerse con el teclado.

```
// Si creas un modal, asegúrate de cerrarlo con Escape  
window.addEventListener('keydown', (e) => {  
  // Ignoramos si el usuario mantiene presionada la tecla (repeat)  
  if (e.repeat) return;  
  if (e.key === 'Escape') cerrarModal();  
});
```

7.5. Delegación de Eventos (El Patrón Rey)

Aprovechando el **Burbujeo**, en lugar de agregar **1,000 event listeners** a **1,000 filas** de una tabla, agregamos un **solo listener** a la **tabla (padre)**.

```
const tabla = document.querySelector('#tabla-usuarios');  
  
tabla.addEventListener('click', (e) => {  
  // closest() busca hacia arriba al ancestro más cercano que sea un botón de borrar  
  const btnBorrar = e.target.closest('.btn-borrar');  
  
  if (!btnBorrar) return; // Si no clickeó en un botón de borrar, salimos  
  
  // matches() comprueba si el botón está desactivado  
  if (btnBorrar.matches('.disabled')) return;  
  
  const id = btnBorrar.dataset.id;  
  borrarUsuario(id);  
});
```


7.6. Rendimiento: Throttle, Debounce y Opciones Pasivas

Los eventos como scroll, mousemove o input se disparan cientos de veces por segundo. Si realizas peticiones fetch o cálculos pesados en ellos, la aplicación colapsará.

1. Debounce (Rebote): "Espera a que el usuario termine de hacer lo que está haciendo antes de ejecutar la función". Es ideal para barras de búsqueda (espera a que deje de tipear para hacer el fetch).

```
function debounce(func, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer); // Reinicia el reloj con cada tecla
    timer = setTimeout(() => { func.apply(this, args); }, delay);
  };
}

const buscar = debounce((texto) => console.log(`Buscando en API: ${texto}`), 500);
inputBuscador.addEventListener('input', (e) => buscar(e.target.value));
```

2. Opciones de AddEventListener (passive y once):

```
// 'once: true' remueve el listener automáticamente después del primer clic
botonPromocion.addEventListener('click', () => alert('Solo una vez!'), { once: true });

// 'passive: true' le promete al navegador que NUNCA usarás preventDefault()
// Permite que el navegador haga un scroll fluido a 60fps sin esperar a tu JS
window.addEventListener('scroll', chequearVisibilidad, { passive: true });
```

7.7. Fugas de Memoria (Memory Leaks) y AbortController

Un **Memory Leak** ocurre cuando se **destruye** un componente **visual** del **DOM**, pero sus **eventos** continúan **escuchando** en **memoria** de forma innecesaria.

Forma tradicional (**Laboriosa**):

Para poder usar `removeEventListener`, es necesario evitar las **funciones anónimas**. Se debe **guardar la referencia exacta** a la función del evento.

```
function manejarClic() { console.log('Clic'); }

btn.addEventListener('click', manejarClic);
// Más tarde, cuando borremos el botón del DOM...
btn.removeEventListener('click', manejarClic);
```

La Forma Moderna 2026 (**AbortController**):

Si tienes 5 listeners en un componente, puedes apagarlos todos juntos con un solo controlador.

```
const controller = new AbortController();

btnX.addEventListener('click', () => console.log('X'), { signal: controller.signal });
btnY.addEventListener('mousemove', () => console.log('Y'), { signal: controller.signal });

// Cuando destruimos el componente:
controller.abort(); // ¡Apaga TODOS los listeners vinculados a este signal de golpe!
```

7.8. Custom Events (Desacoplar Lógica)

A veces quieres que componentes distantes se comuniquen sin depender de un archivo central.

```
// Creamos y disparamos un evento personalizado inventado por nosotros
const loginExitoso = new CustomEvent('app:login', { detail: { username: "Ana" } });
document.dispatchEvent(loginExitoso);

// En otro archivo completamente distinto:
document.addEventListener('app:login', (e) => {
  console.log(`Bienvenida, ${e.detail.username}`);
});
```

7.9. Errores Comunes en Eventos

1. **Olvidar el `preventDefault()`**: El formulario se envía y la página se recarga en milisegundos, borrando toda tu magia de JS y dejando la consola en blanco.
2. **Delegación fallida por `target`**: Si tu botón tiene un icono de FontAwesome adentro, `e.target` será el icono, no el botón. Si intentas leer el `dataset` en `e.target`, será **undefined**. **Solución**: Usa siempre `e.target.closest('.tu-boton')`.
3. **Pérdida de Rendimiento** (Pesticida de Memoria): Usar **arrow functions** dentro de un bucle para agregar eventos a componentes que luego se eliminan del DOM, sin apagar nunca el evento (causa degradación silenciosa del rendimiento).

8. Asincronía y Fetch API

La **asincronía** es el núcleo de cualquier aplicación **web moderna**. Si **JavaScript** bloqueara la pantalla cada vez que descarga **datos**, la web sería **inusable**. Aquí aprenderemos a dominar la **comunicación con el exterior**.

8.1. El Modelo Asíncrono y el Event Loop (Repaso aplicado)

Recordemos la arquitectura de **JS**: el **Call Stack** ejecuta el código **síncrono**. Cuando usamos **fetch()** (una **Web API**), el navegador delega esa tarea a la red. Una vez que los datos llegan, la respuesta no va al **Call Stack** directamente; viaja a la **Microtask Queue** (la fila **VIP**), esperando que el código actual termine de ejecutarse.

Orden de ejecución (Comprendiendo la asincronía):

```
console.log('1. Síncrono');

// Macrotask (Fila normal)
setTimeout(() => console.log('4. Macrotask (Timer)'), 0);

// Microtask (Fila VIP)
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(() => console.log('3. Microtask (Fetch resuelto)'));

console.log('2. Síncrono');
// Salida: 1 -> 2 -> 3 -> 4
// Aunque la red tarde milisegundos, JS siempre ejecuta el código síncrono primero.
```

8.2. Promesas en Profundidad

Una Promesa es un objeto que representa la eventual finalización (o fallo) de una operación asíncrona.

.then/.catch vs async/await:

- **async/await (Recomendado 2026)**: Hace que el código **asíncrono** se lea como código secuencial **síncrono**. Úsalo para el **95%** de tus flujos lógicos.
- **.then()**: Sigue siendo **útil** para operaciones simples en una sola línea o cuando no estás dentro de una función **async**.

Combinadores de Promesas (Casos de Uso Reales):

A veces no queremos esperar a una petición para empezar otra. Queremos lanzarlas en **paralelo**.

- **Promise.all([p1, p2]): "Todo o nada"**. Lanza varias peticiones a la vez y espera a que **todas terminen**. Si **una falla, todo el bloque falla**. Ideal para cargar los **datos críticos iniciales** de una vista.
- **Promise.allSettled([p1, p2]): "Tolerancia a fallos"**. Espera a todas, pero si una falla, **no importa**, devuelve un *array* detallando cuáles tuvieron **éxito** y cuáles **no**.
- **Promise.race([p1, p2]): "El más rápido"**. Devuelve el resultado de la **primera promesa que termine** (ya sea éxito o error). Se usaba históricamente para forzar *timeouts*.
- **Promise.any([p1, p2]):** Devuelve la **primera que tenga éxito**, ignorando las fallidas.

Errores Típicos con Promesas:

- **"Olvidar el await"**: Si declaras `const data = fetch(...)` sin `await`, tu variable `data` guardará un objeto de tipo **Promise {<pending>}**, **no la información JSON** que esperabas.
- **"Unhandled Rejection"**: Ocurre cuando una **promesa falla** (ej. error de red) y no escribiste un bloque `.catch()` (o `try/catch` con `async/await`). El navegador lanza una **advertencia severa**.
- **"Async sin Return"**: Toda función marcada con `async` devuelve **automáticamente una Promesa**. Si olvidas el `return` dentro de ella, devolverá **Promise<undefined>**.

8.3. Fetch API

La función `fetch()` es la **función nativa** para realizar **peticiones HTTP**. Es fundamental comprender cómo gestionar sus **respuestas**.

El detalle crítico: Error de Red vs Error HTTP (`res.ok`).

Si se intenta solicitar datos y el usuario carece de conexión a internet, `fetch` activará el bloque `catch` (Error de Red). SIN EMBARGO, si el servidor responde con un código de error (por ejemplo, 404 Not Found o 500 Internal Error), `fetch` considera la conexión exitosa y NO activará el `catch`.

Para solventar esta situación, es imprescindible comprobar la propiedad booleana `res.ok` (que es `true` si el código HTTP está en el rango 200-299).

Helper Reutilizable (`fetchJson`):

Crea esta **función** en tus proyectos para **evitar repetir el control de errores** en cada **petición**.

```
async function fetchJson(url, options = {}) {
  const response = await fetch(url, options);

  // Si el servidor responde con error (4xx o 5xx), forzamos la explosión al catch
  if (!response.ok) {
    throw new Error(`Error HTTP: ${response.status} - ${response.statusText}`);
  }

  return await response.json(); // Parsea el body de la respuesta
}
```

Tipos de Peticiones (GET, POST, PUT, DELETE):

- **GET** (Por defecto): Solicita **información**. Los **datos** van en la **URL (Query Params)**.
- **POST**: **Envía** información **nueva** al servidor (ej. **crear** un usuario). Requiere **body**.
- **PUT/ PATCH**: **Actualiza** información **existente**.
- **DELETE**: Solicita **borrar** un **recurso**.

Ejemplo 1: GET con Query Params Seguros (`URLSearchParams`)

Nunca construyas URLs concatenando strings (`?q= + texto`), ya que puede generar problemas con caracteres especiales (espacios, `&`).

```
const url = new URL(['https://api.ejemplo.com/usuarios',
  (https://api.ejemplo.com/usuarios)']);
url.searchParams.append('rol', 'admin');
url.searchParams.append('busqueda', 'juan p rez'); // El espacio se codificar  solo

// fetch(url) internamente usar : "[https://api.ejemplo.com/usuarios?
rol=admin&busqueda=juan+p rez](https://api.ejemplo.com/usuarios?
rol=admin&busqueda=juan+p rez)"
```

Ejemplo 2: **POST JSON** con **Headers**


```
const nuevoUsuario = { nombre: 'Ana', edad: 28 };

fetch(['https://api.ejemplo.com/usuarios'](https://api.ejemplo.com/usuarios'), {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json', // Le decimos al backend que le mandamos JSON
    'Accept': 'application/json',      // Le decimos al backend qué esperamos de vuelta
    'Authorization': 'Bearer mi-token' // Token JWT para seguridad
  },
  body: JSON.stringify(nuevoUsuario) // Fetch no convierte el objeto, debes usar stringify
});
```

Ejemplo 3: Subida de Archivos con **FormData**

Si necesitas subir una **imagen** o **archivo**, no se utiliza **JSON**. En su lugar, el navegador provee el objeto **FormData**, el cual se encarga de preparar los **archivos** correctamente para el envío (y automáticamente configura el **Content-Type** adecuado).

```
const formNode = document.querySelector('#mi-formulario');
const formData = new FormData(formNode); // Extrae todos los inputs automáticamente

fetch(['https://api.ejemplo.com/upload'](https://api.ejemplo.com/upload'), {
  method: 'POST',
  body: formData // No usas JSON.stringify, le pasas el objeto FormData directo
});
```

8.4. AbortController, Timeouts y Cancelaciones

A veces el servidor se queda **"pensando"** y la app queda **colgada**. Otras veces, el usuario hace una **búsqueda** ("A", luego "An", luego "Ana") y las **peticiones de red** llegan en **desorden**, actualizando la pantalla con **resultados viejos** (**Condición de carrera / Race Condition**).

Para solucionar ambas cosas, usamos **AbortController**.

1. Cancelación Automática por Tiempo (Timeout):

El estándar 2026 provee `AbortSignal.timeout()`.

```
// La petición se cortará violentamente si tarda más de 5000ms
const res = await fetch(url, { signal: AbortSignal.timeout(5000) });

Fallback para motores antiguos:
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);

const res = await fetch(url, { signal: controller.signal });
clearTimeout(timeoutId); // Limpiamos el timer si la petición tuvo éxito antes
```


2. Cancelación por Búsqueda en Vivo (Live Search):

```
Aquí combinamos el AbortController global con nuestro input.
let currentAbortController = null;

async function buscarUsuarios(query) {
  // 1. Si hay una petición en vuelo, la "asesinamos"
  if (currentAbortController) {
    currentAbortController.abort();
  }

  // 2. Creamos un controlador nuevo para esta nueva petición
  currentAbortController = new AbortController();

  try {
    const url = new URL('https://api.ejemplo.com/search')
    url.searchParams.append('q', query);

    // Le pasamos la señal a fetch
    const res = await fetch(url, { signal: currentAbortController.signal });
    if (!res.ok) throw new Error('Error en búsqueda');

    const data = await res.json();
    renderizarUI(data); // Solo se renderiza si la petición sobrevivió
  } catch (error) {
    if (error.name === 'AbortError') {
      console.log('Petición vieja abortada por el usuario (normal).');
    } else {
      console.error('Error real de servidor o red:', error);
    }
  }
}
```

8.5. CORS Explicado para Frontend

CORS (Cross-Origin Resource Sharing) es un error clásico en el desarrollo web.

Analogía: Eres un **ciudadano** (Frontend en `localhost:3000`) intentando entrar a un **edificio extranjero** (Backend en `api.ejemplo.com`). El **guardia de seguridad** de la puerta (el **Navegador Web**) te detiene y le pregunta al edificio: "Oye, ¿este extranjero tiene permiso para entrar?". Si el edificio no responde **afirmativamente**, el guardia **bloquea** tu **petición**.

El Preflight (OPTIONS):

Para peticiones complejas (como POST con JSON o con tokens Authorization), el navegador lanza una petición "fantasma" invisible llamada OPTIONS para preguntarle al servidor si está de acuerdo. Si el preflight falla, tu `fetch` original es bloqueado con el temido error CORS de consola.

¿Cómo se soluciona?

- Es un problema del Backend: El desarrollador backend debe configurar sus cabeceras (`Access-Control-Allow-Origin: *`).
- ¿Qué hace el Frontend mientras tanto? Puedes configurar tu empaquetador (ej. Vite Proxy) para que actúe como "puente". Tu frontend le pide los datos a tu propio servidor Vite (`localhost:3000/api`), y Vite (que no es un navegador y no tiene restricciones CORS) le pide los datos al backend real.

8.6. Buenas Prácticas de UX (Experiencia de Usuario)

- **Estados de la UI** (Loading, Error, Empty): Toda **petición asíncrona** debe mostrar un **Spinner o Skeleton (Cargando)** al inicio. Si falla el `catch`, debe mostrar el **mensaje de Error**. Si el array JSON llega vacío (`data.length === 0`), debe mostrar un mensaje **"No hay resultados"**. ¡Evita spinners infinitos!
- **Idempotencia**: Es crucial entender qué peticiones son **seguras de reintentar** si fallan. Reintentar un **GET** (obtener clima) es seguro (**es idempotente**). Reintentar un **POST** (comprar un producto) a ciegas si falló la red es **peligroso**, ya que se podría **cobrarle dos veces al usuario**.
- **Mensajes Amigables**: **Jamás** le imprimas al usuario el `error.message` técnico (`Failed to fetch`). Muestra mensajes como: **"Parece que tu conexión es inestable. Revisa tu internet y vuelve a intentar."**

8.7. Seguridad Mínima en Fetch

1. **Tokens Visibles**: Jamás loguees (`console.log`) objetos que contengan tokens JWT de autenticación o contraseñas.
2. **Sanitización del DOM**: Recordatorio vital. Los datos JSON que traes del `fetch` deben inyectarse en el HTML usando `textContent`, nunca `innerHTML`. Si un usuario guardó código `<script>` malicioso en la base de datos, al leerlo e inyectarlo con `innerHTML`, provocarás un ataque XSS en todos tus demás usuarios.

9. Web Storage API y Privacidad (Cookies)

En **aplicaciones frontend robustas**, a menudo necesitamos "recordar" **información** en el **dispositivo del usuario** para evitar solicitarla constantemente al **servidor**. Aquí es donde las **APIs de almacenamiento del navegador** y las **Cookies** desempeñan un papel crucial.

9.1. El Modelo Mental y Reglas de Oro

¿Qué **SÍ** guardar en el **Web Storage**?

- **Preferencias no sensibles** de la **interfaz** (modo oscuro/claro, idioma).
- **Caché de datos públicos** para **evitar peticiones repetitivas** (ej. listado de categorías).
- **Estado temporal** de un **formulario** que el usuario está llenando pero aún no envió.

¿Qué **NO** guardar **NUNCA** en el **Web Storage**?

- Contraseñas.
- Tokens JWT de autenticación (AccessToken) o Refresh Tokens.
- Información personal sensible (tarjetas de crédito, datos médicos).

Alerta de Seguridad (XSS): Cualquier **script de JavaScript** que corra en tu página puede **leer localStorage y sessionStorage**. Si tu sitio sufre una **inyección XSS** (ej. renderizaste un comentario de un usuario con innerHTML), el **atacante** puede **extraer tus tokens** y enviarlos a su propio servidor en 1 milisegundo.

9.2. LocalStorage vs SessionStorage

Ambas interfaces comparten exactamente los mismos métodos, pero difieren en su persistencia y alcance.

Característica	sessionStorage (Sesión)	localStorage (Local)
Persistencia	Temporal: Se borra automáticamente al cerrar la pestaña o ventana.	Permanente: Solo se borra por código o cuando el usuario limpia sus datos de navegación.
Alcance	Por pestaña: Dos pestañas idénticas no comparten su sessionStorage.	Por origen (dominio): Todas las pestañas del mismo sitio comparten el mismo Storage.
Capacidad	~5 MB por origen.	~5 a 10 MB por origen.

9.3. Robustez y Errores Típicos

Trabajar con **Storage** parece fácil (**setItem** y **getItem**), pero en **producción** hay **casos borde** que romperán tu aplicación si no los manejas:

1. Manejo de null en getItem: Si la clave no existe, getItem devuelve null.
2. Try/Catch en JSON: Web Storage solo guarda Strings. Siempre debes usar JSON.stringify() al guardar y JSON.parse() al leer. Si los datos están corruptos, JSON.parse lanza un error fatal.
3. Límite de Cuota (QuotaExceededError): Si intentas guardar más de 5MB, el navegador lanzará una excepción.

Mini Práctica: Helpers de Almacenamiento Seguro

Crea funciones contenedoras (wrappers) para que tu código sea a prueba de balas:

```
// Guardar de forma segura
function writeJsonStorage(key, value) {
  try {
    const stringValue = JSON.stringify(value);
    localStorage.setItem(key, stringValue);
  } catch (error) {
    if (error.name === 'QuotaExceededError') {
      console.error('El Storage está lleno. Necesitas limpiar la caché.');
```

```
      // Aquí podrías ejecutar una limpieza: localStorage.clear();
    }
  }
}

// Leer de forma segura (con valor por defecto)
function readJsonStorage(key, fallback = null) {
  try {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : fallback;
  } catch (error) {
    console.error(`Error parseando el item ${key} del storage`, error);
    return fallback;
  }
}

// Uso:
const preferencias = readJsonStorage('ui_prefs', { theme: 'light' });
```

9.4. Sincronización entre Pestañas (Evento 'storage')

Una característica **poderosa** y poco conocida de **localStorage** es que emite un **evento** llamado **storage** solo en las demás pestañas del mismo dominio cuando hay un **cambio**. Es **ideal para sincronizar el estado global** de la aplicación.

Ejemplo Práctico: Logout Global o Modo Oscuro

Imagina que el usuario tiene 3 pestañas de tu app abiertas. En la pestaña 1, hace clic en "Cerrar Sesión". Quieres que las pestañas 2 y 3 también reaccionen inmediatamente.

```
// En todas las pestañas, escuchamos el evento 'storage' de window
window.addEventListener('storage', (event) => {
  // Este evento tiene propiedades muy útiles
  if (event.key === 'app_logout') {
    alert('Has cerrado sesión en otra pestaña. Redirigiendo...');
    window.location.href = '/login';
  }

  if (event.key === 'ui_theme') {
    aplicarModoOscuro(event.newValue === 'dark');
  }
});

// En la Pestaña 1, cuando el usuario hace clic:
btnCerrarSesion.addEventListener('click', () => {
  localStorage.setItem('app_logout', Date.now()); // El cambio dispara el evento en las
  otras tabs
});
```

9.5. Alternativas de Almacenamiento Modernas

Si 5MB de strings no son suficientes, la web ofrece otras **APIs nativas**:

- IndexedDB: Una base de datos NoSQL completa dentro del navegador. Soporta cientos de Megabytes, transacciones, y almacenamiento de archivos binarios (Imágenes, Blobs). Es asíncrona.
- Cache Storage API: Usada principalmente junto con los Service Workers para construir Progressive Web Apps (PWA) que funcionen sin conexión a internet (modo offline), cacheando respuestas HTTP enteras.

9.6. Cookies a Fondo

Las Cookies son pequeños fragmentos de datos (~4KB) que viajan automáticamente al servidor en cada petición HTTP. A diferencia del `localStorage`, el propósito principal de las cookies no es almacenar datos para el Frontend, sino recordarle al Servidor quién eres (gestión de sesión).

¿Cómo funciona una Cookie de Sesión Segura?

Normalmente, el Frontend no crea la cookie. El Backend, tras validar el login, envía una cabecera HTTP: `Set-Cookie: sessionId=123xyz; HttpOnly; Secure; SameSite=Strict;`

Atributos **Vitales** de las Cookies:

- **HttpOnly**: La **regla de oro de la seguridad**. Si una cookie tiene este *flag*, **JavaScript no puede leerla** (ni con `document.cookie`). Esto te protege completamente del **robo de sesiones** vía **ataques XSS** (Cross-Site Scripting).
- **Secure**: Obliga a que la cookie solo viaje si la conexión es **HTTPS** (**cifrada**).
- **SameSite**: Protege contra **ataques CSRF** (Cross-Site Request Forgery).
- **Strict**: La cookie solo viaja si navegas dentro del **propio sitio**.
- **Lax**: (Por defecto) Viaja en navegaciones de **nivel superior** (ej. al seguir un link desde otro sitio hacia el tuyo).
- **None**: Permite enviar la cookie desde **sitios de terceros** (requiere obligatoriamente **Secure**).
- **Expires** o **Max-Age**: Define **cuándo muere la cookie**. Si no se especifican, se considera una "**Cookie de Sesión**" y muere al **cerrar el navegador**.

9.7. Privacidad Moderna: CHIPS y Cookies Particionadas

Las **Cookies de Terceros** (*Third-Party Cookies*) son aquellas creadas por un **dominio diferente** al que estás visitando (ej. un widget de Facebook insertado en tu blog). Históricamente, se usaron para **rastrear usuarios** a través de todo internet y mostrar **publicidad dirigida**.

Desde **2026**, los **navegadores** (**Safari, Firefox y Chrome**) **bloquean por defecto** las cookies de terceros por motivos de **privacidad**. No obstante, esto generó problemas con **usos legítimos** (como un *iFrame* de un chat de soporte).

La **solución** es **CHIPS** (*Cookies Having Independent Partitioned State*), el **nuevo estándar**. Si necesitas que una cookie funcione en un **entorno incrustado** (ej. dentro de un `<iframe>` en un sitio ajeno), debes agregar el **atributo Partitioned**.

Esto crea un "**doble candado**": la cookie queda atada tanto al **dominio incrustado** como al **dominio "padre"** que lo está mostrando. De esta manera, el widget funciona, pero es **imposible usar esa cookie para rastrear** al usuario cuando visite otra página.

9.8. ¿Dónde guardo este dato?

Regla de decisión rápida para arquitecturas Frontend:

1. ¿Es un Token de Autenticación (JWT/Sesión)?
 - Cookie HttpOnly y Secure. Que la maneje el navegador y el backend; el Frontend JS ni siquiera debería tocarla.
2. ¿Es una preferencia de Interfaz (Modo oscuro, filtros)?
 - localStorage. Queremos que persista cuando el usuario vuelva mañana.
3. ¿Es el estado de un formulario largo o un carrito temporal no crítico?
 - sessionStorage. Si el usuario cierra la pestaña, asumimos que abandonó el proceso.
4. ¿Son Megabytes de datos estructurados para uso Offline (PWA)?
 - IndexedDB o Cache Storage.

10. Práctica Integradora: "TaskFlow Profesional"

Para consolidar la **Unidad 1**, construiremos **TaskFlow Profesional**, un **gestor de tareas** que no es solo un script básico, sino una **aplicación modular** diseñada con los **estándares de 2026**.

Arquitectura de la Práctica:

En lugar de un archivo "espagueti", aplicaremos una **arquitectura limpia (Módulos ES)**:

- **api.js**: Simulará nuestro servidor (**asincronía, promesas**).
- **storage.js: Wrappers** para guardar la **caché local**.
- **dom.js**: **Renderizado eficiente** con `<template>` y **DocumentFragment**.
- **app.js**: **Orquestador principal** y **delegación de eventos**.

Paso 0: Setup Profesional del Proyecto

No abriremos un bloc de notas. Usaremos el **ecosistema Node**.

1. Crear el **proyecto** con **Vite**:

```
npm create vite@latest taskflow -- --template vanilla  
cd taskflow
```

2. Instalar **dependencias** (**Tailwind**):

```
npm install -D tailwindcss postcss autoprefixer
```

3. Iniciar **Tailwind**:

```
npx tailwindcss init -p
```

Configura tu **tailwind.config.js** para que lea tus archivos: **content**:
`['./index.html', './src/**/*.js']`.

En tu package.json observarás los scripts de Vite:

- **"dev"**: `"vite"` -> Levanta tu **servidor local** (Hot Reload).
- **"build"**: `"vite build"` -> **Empaqueta, minifica y purga el CSS** para producción.

Paso 1: HTML Semántico y Accesible (index.html)

Diseñaremos una UI con estados (Loading, Empty, Error) y accesibilidad total.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>TaskFlow Pro</title>
  <link rel="stylesheet" href="./src/style.css">
</head>
<body class="bg-slate-50 text-slate-900 min-h-screen p-8">

  <main class="max-w-2xl mx-auto bg-white p-6 rounded-xl shadow-lg">
    <header class="mb-6">
      <h1 class="text-3xl font-bold">TaskFlow Pro</h1>
    </header>

    <!-- Formulario Accesible -->
    <form id="task-form" class="flex gap-3 mb-6">
      <label for="task-input" class="sr-only">Nueva tarea</label>
      <input
        type="text"
        id="task-input"
        required
        placeholder="Ej. Estudiar Arquitectura JS ..."
        class="flex-1 border-2 border-slate-200 rounded-lg p-3 focus:outline-none
focus:border-blue-500 focus:ring-1 focus:ring-blue-500 transition-all"
      >
      <!-- IMPORTANTE: type="submit" para que funcione con Enter -->
      <button type="submit" class="bg-blue-600 hover:bg-blue-700 text-white px-6 py-3 rounded-
lg font-medium transition-colors">
        Añadir
      </button>
    </form>

    <!-- Estados de la UI -->
    <div id="ui-state" class="text-center py-4 text-slate-500 hidden" aria-live="polite">
      <!-- Aquí inyectaremos "Cargando ..." o "No hay tareas" -->
    </div>

    <!-- Contenedor de la lista -->
    <ul id="task-list" class="space-y-3"></ul>

    <!-- Template Inerte para clonar con JS -->
    <template id="task-template">
      <li class="group flex justify-between items-center p-4 bg-slate-50 rounded-lg border
border-slate-100 hover:border-blue-200 transition-all">
        <div class="flex items-center gap-3">
          <input type="checkbox" class="task-check w-5 h-5 cursor-pointer accent-blue-600"
aria-label="Marcar como completada">
          <span class="task-title text-lg transition-all duration-300"></span>
        </div>
        <button type="button" class="btn-delete text-slate-400 hover:text-red-500 focus-
visible:ring-2 focus-visible:ring-red-500 rounded p-1 transition-colors" aria-label="Borrar
tarea">
          <svg class="w-6 h-6 pointer-events-none" fill="none" stroke="currentColor"
viewBox="0 0 24 24"><path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M19 7L-.867 12.142A2 2 0 0116.138 21H7.862a2 2 0 01-1.995-1.858L5 7m5 4v6m4-6v6m1-10V4a1 1
0 00-1-1h-4a1 1 0 00-1 1v3M4 7h16"></path></svg>
        </button>
      </li>
    </template>
  </main>

  <script type="module" src="./src/app.js"></script>
</body>
</html>

```

Paso 2: Persistencia y "Fake API" (src/api.js y src/storage.js)

Para no depender de un servidor externo, crearemos un Mock (simulador) de la API usando Promesas y setTimeout. Almacenaremos la caché en localStorage.

```
// src/storage.js - Módulo puro para LocalStorage
const STORAGE_KEY = 'taskflow_data';

export const storage = {
  get: () => {
    try {
      const data = localStorage.getItem(STORAGE_KEY);
      return data ? JSON.parse(data) : [];
    } catch (e) {
      console.error("Error leyendo storage", e);
      return [];
    }
  },
  set: (data) => {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
  }
};

// src/api.js - Simulador asíncrono
import { storage } from './storage.js';

// Retardo simulado de red (500ms)
const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms));

export const fakeApi = {
  async getTasks(signal) {
    await delay(500); // Simulamos la red

    if (signal?.aborted) throw new DOMException('Aborted', 'AbortError');
    // En el mundo real, aquí habría un fetch(). En su lugar, leemos la "BD" local.
    return storage.get();
  },
  async saveTask(task) {
    await delay(300); // Guardando en el servidor imaginario...
    const currentTasks = storage.get();
    currentTasks.push(task);
    storage.set(currentTasks);
    return task;
  },
  async deleteTask(taskId) {
    await delay(300);
    const tasks = storage.get().filter(t => t.id !== taskId);
    storage.set(tasks);
    return taskId;
  },
  async toggleTask(taskId, isCompleted) {
    await delay(200);
    const tasks = storage.get().map(t =>
      t.id === taskId ? { ...t, completed: isCompleted } : t
    );
    storage.set(tasks);
  }
};
```

Paso 3: Renderizado Eficiente del DOM (src/dom.js)

Separamos la lógica visual de la lógica de negocio. Usaremos DocumentFragment y replaceChildren para evitar el temido Layout Thrashing.

```
// src/dom.js
const listEl = document.querySelector('#task-list');
const template = document.querySelector('#task-template');
const uiStateEl = document.querySelector('#ui-state');

export const dom = {
  renderState(state, message = '') {
    if (state === 'loading') {
      uiStateEl.textContent = '⌚ Cargando tareas ...';
      uiStateEl.classList.remove('hidden');
    } else if (state === 'empty') {
      uiStateEl.textContent = '🌟 ¡Todo al día! No hay tareas.';
      uiStateEl.classList.remove('hidden');
    } else {
      uiStateEl.classList.add('hidden');
    }
  },

  renderTasks(tasksArray) {
    if (tasksArray.length === 0) {
      listEl.replaceChildren(); // Vacía rápido
      this.renderState('empty');
      return;
    }

    this.renderState('ready');
    const fragment = document.createDocumentFragment();

    tasksArray.forEach(task => {
      // Clon profundo del template
      const clone = template.content.cloneNode(true);
      const li = clone.querySelector('li');
      const title = clone.querySelector('.task-title');
      const checkbox = clone.querySelector('.task-check');
      const btnDelete = clone.querySelector('.btn-delete');

      // Seguridad: textContent para evitar XSS
      title.textContent = task.title;

      // Estado visual si está completada
      if (task.completed) {
        checkbox.checked = true;
        title.classList.add('line-through', 'text-slate-400');
        li.classList.replace('bg-slate-50', 'bg-slate-100/50');
      }

      // Dataset para la delegación de eventos
      btnDelete.dataset.id = task.id;
      checkbox.dataset.id = task.id;

      fragment.append(clone);
    });

    // Inserción única y eficiente en el DOM real
    listEl.replaceChildren(fragment);
  }
};
```

Paso 4: El Orquestador (src/app.js)

Atamos todo usando Delegación de Eventos y AbortController.

```
// src/app.js
import { fakeApi } from './api.js';
import { dom } from './dom.js';

// Estado global de la vista (Caché local)
let currentTasks = [];
let loadController = null;

const form = document.querySelector('#task-form');
const input = document.querySelector('#task-input');
const listEl = document.querySelector('#task-list');

// 1. Inicialización de la app
async function initApp() {
  dom.renderState('loading');
  loadController = new AbortController();

  try {
    currentTasks = await fakeApi.getTasks(loadController.signal);
    dom.renderTasks(currentTasks);
  } catch (error) {
    if (error.name === 'AbortError') {
      dom.renderState('error', 'Fallo al cargar las tareas.');
```


Paso 5: Debugging (DevTools)

Una vez implementado, utiliza las **Chrome DevTools** para auditar tu propio trabajo:

- **Network** (Red): Cambia a **Slow 3G** para ver cómo funciona el estado **loading**.
- **Application** (Almacenamiento): Abre **Local Storage**, añade una tarea y verifica que el **JSON** se guarda con formato correcto.
- **Console** (Consola): Escribe una etiqueta `alert('XSS')` en el input. Asegúrate de que **no se ejecute** (comprobando que usaste `textContent`).
- **Accesibilidad** (Teclado): Navega por tu App usando solamente el **tabulador** (Tab) y la **barra espaciadora**. ¿Se **resaltan los botones** al pasar sobre ellos?

11. Errores Comunes de Desarrollo

1. **El Formulario Zombi**: Olvidar el `e.preventDefault()` en el `submit`. La app se recarga, perdiendo todo el **estado de JS** y regresando a 0.
2. **Botones Invisibles al Teclado**: Usar `<div class="btn">` en lugar de `<button type="button">`. El **usuario de teclado** no puede borrarlas.
3. **Riesgo XSS con Template Strings**: Construir la lista usando `list.innerHTML += <li>${input.value}</li>`. Es un **fallo grave de seguridad**. Usar `<template>` y `textContent` nos salva de esto.
4. **Pérdida de Contexto en Eventos**: Intentar leer `e.target.dataset.id` cuando el usuario hizo clic en el ícono `<svg>` dentro del botón de borrar (el SVG no tiene ID). Solución: Siempre usar `e.target.closest('.btn-delete')`.
5. **No Serializar el Storage**: Intentar guardar directamente `localStorage.setItem('tasks', currentTasks)`. El navegador guardará `"[object Object]"`. Siempre usa `JSON.stringify`.
6. **El Terror del null en Storage**: Hacer `JSON.parse(localStorage.getItem('tareass'))` la primera vez que se abre la app (cuando no existe la llave) devolverá un **error**. Siempre prevé un **valor por defecto** (Fallback `|| []`).

7. **Peticiones Fantasma (Memory Leaks):** En una App real, si el usuario navega fuera de la lista de tareas antes de que el `getTasks()` termine, debes **cancelar la petición** con `AbortController`.
8. **Layout Thrashing** al pintar: Hacer un `for` de 100 tareas y por cada una hacer `listEl.appendChild()`. Redibuja la pantalla 100 veces seguidas, **congelando la interfaz**. Usa `DocumentFragment` y dibuja **1 sola vez**.

Atribución-No Comercial-Sin Derivadas



Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera:

Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.